

Department of Applied Data Science

DATA 298B MSDA Project II

Workbook 2

Team 2

**Basanth Periyapatna Roopa Kumar, Manav Rajesh Anandani,
Nandhakumar Apparsamy, Nischitha Nagendran, Nitya Rondla**

03/22/2025

4. Model Development

This part indicates the model development plan of the Personalized Education Agent. The main aim of the project is to create a smart academic support system that has the capacity to explain technical terms and concepts, help students in their coding systems, in debugging, and provide structured reasoning to complex academic questions. In order to achieve this goal, the project considers a variety of open-weight large language models, instead of resorting to one paradigm. This would allow the system to balance the quality of responses, level of reasoning, coding skills, and efficiency of deployment.

There were three main priorities that guided the model development process. To begin with, the selected models should be able to provide educational responses as opposed to completing generic text. Second, they have to be fine-tunable and accessible with practical limitations of academic computer resources. Third, they ought to form part and parcel of a system architecture that allows the selective implementation of various models of divergent tutoring tasks to be operated. Due to these requirements, eight concrete models were chosen within five key families: Mistral-7B-Instruct-v0.3, Llama-3.2-3B-Instruct, Llama-3.1-8B-Instruct, Qwen 3.5 4B, Qwen 3.5 9B, Gemma-2-2B-it, Gemma-2-9B-it, and DeepSeek-R1 7B Distilled.

All of these models have the basic requirements of the desired system incorporated. Smaller capacity models provide faster response time and lower cost of deployment, thus facilitating lightweight educational interactions; bigger capacity models allow more reasoning and greater explanation power to tasks with a high conceptual demand and high technical complexity. Models that have a strong coding orientation enhance programming support, and models that have a stronger reasoning-oriented orientation enhance performance on several step, higher-order problem-solving prompts. The following subsection has identified the proposed models and elaborated on the reasons behind the incorporation of those models into the development framework.

4.1 Model Proposals

To proceed with the mission of the Personalized Education Agency, the project compiled a faculty of open-weighted, instruction-tuned language models, which show heterogeneous abilities in tutoring, reasoning, coding, and efficient implementation. Instead of picking the models only by the criteria of dimensions, the goal was to establish a comparative frame that is consistent with the dynamics of an educational system. The variety of student questions and the need to provide quick responses, finer code-level assistance, and substantive intellectual considerations; therefore, the selected models were filtered to fulfill these specific teaching needs.

Each of the selected models was evaluated as a potential module in a multi-model tutoring framework. The input given to each model within the system includes a structured academic prompt comprising the student query, contextual information about the subject matter, level of difficulty, and type of task. The expected deliverables include narrative descriptions of the concepts of the explanation, a systematic series of reasoning, code generation, debugging support, and summarized instructions. This consistent, tutoring-based implementation makes it possible to make the models comparable to each other and assign the specific functional roles to the final architecture of the system.

4.1.1 Mistral-7B-Instruct-v0.3

As a powerful instruction-tuned base, mistral-7B-Instruct-v0.3 was selected to provide balanced tutoring results. It presents a practical combination of model capacity, instruction-following behavior, and feasibility to fine-tune. Mistral is a general-purpose educational model used in the present research that can be used to aid the concept elucidation, moderate reasoning problems, and structured response generation without the increased computational expense of other larger systems.

The major virtue of Mistral lies in its balance. It is large enough to produce meaningful instructional reactions and small enough to be practical as an experimental implementation and to be deployed on an academic scale. As a result, Mistral is a central point of reference in the development process of the model, which will help evaluate how far the quality of tutoring is possible using one mid-sized instruction-tuned model.

4.1.2 Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct

The Llama models were added to make a comparative analysis of a short pedagogical variant and a more expanded reasoning-based one belonging to the same family. Llama -3.2-3B-Instruct refers to a small model that is more economical to train and use in short teaching conversations. In its turn, Llama-3.08B-Instruct is a more advanced model that is expected to achieve better results in broad-based explanations, step-based reasoning, and broad-based tutoring.

The relevance of this model pair is that it allows discussing the trade-off between the parametric scale and pedagogical efficacy in the context of a homogeneous architectural taxonomy. A smaller Llama configuration is beneficial in cases of quick conceptual support and resource-saving utilisation, and a bigger configuration is more appropriate in cases of complex explanatory work, where the clarity and richness of the arguments are more important than the low latency concerns. Taken together, the two models provide a solid platform on which to conduct a critical analysis of the differentiated effect of model size on instructional performance in a tutoring scenario.

4.1.3 Qwen 3.5 4B and Qwen 3.5 9B

The Qwen models were mostly chosen in technical tutoring and work involving programming. Qwen 3.5 4B is the smaller and more productive, and Qwen 3.5 9B is more productive with better capabilities of richer coding support and more robust technical arguments. The Personalized Education Agent will be particularly applicable to the project due to the relevance of this model line that will assist with the help in coding, debugging, and explaining the algorithmic logic.

The usefulness of the Qwen models is that they are appropriate to structured code-centered interaction. They are not considered mere code generators in this project. They are suggested as tutoring models that may explain the logic of implementation, detect probable coding errors, and offer support that is technically significant and presented in a manner comprehensible by the learners. The 4B model promotes efficiency-based experimentation, and the 9B model boosts performance on program tasks that are more challenging in nature.

4.1.4 Gemma-2-2B-it and Gemma-2-9B-it

The Gemma models were chosen to test the performance of the lightweight and bigger instruction-tuned models in an educational environment. Gemma-2-2B, it can be considered a compact and deployment-friendly model that can provide prompt academic support and easy tutoring feedback. Gemma-2-9B - it is a larger variant, expected to generate better reasoning quality and more broadly-educational output without going above and beyond an open-weight, instruction-tuned architecture.

This pair of models is especially useful in defining the trade-off between efficiency and ability in the ultimate system. The smaller Gemma variant is easier to use in interactions with low latency and in resource-constrained deployment, but the larger version is capable of more serious reasoning and explanation. The combination of the two versions makes the report even stronger by showing that the development of models was based on the focus not only on the peak capability but also on a realistic system design.

4.1.5 DeepSeek-R1 7B Distilled

DeepSeek-R1 7B Distilled was selected as the reasoning-based model in the development architecture. Its main role is to support more technically challenging educational prompts that require logical dissection, deep analysis, and a more systematic problem-solving paradigm. In turn, it is particularly beneficial when it comes to explaining complicated ideas and performing multi-step technical reasoning.

The importance of this model in the project can be explained by the fact that educational assistance is not limited to short replies or lay-level explanations. Some of the questions posed by a student will require a more strict line of reasoning and a detailed breakdown of the issue. In this regard, DeepSeek-R1 7B Distilled was added to determine the degree to which a reasoning-based model can be used to improve tutoring efficacy under such demanding situations. It identifies the sophisticated line of reasoning in the culminating system architecture.

4.1.6 Proposed Role of the Models in the System

The chosen models were not suggested as individual solutions; instead, they were selected as parts of a complex educational AI system. Mistral-7B-Instruct-v0.3 is an instruction-tuned balanced baseline. The Llama-3.2-3B-Instruct and Gemma-2-2B-it can handle efficient tutoring environments with fewer resource requirements. Llama318BInstruct, Gemma29Bit, and DeepSeak R17B Distilled all help to provide more explanations and greater reasoning abilities. Qwen 3.5-4B and Qwen 3.5-9B will improve the programming and debugging expertise of the system.

This is a strength of the project as it is a fundamental model selection strategy. The system will not force one model to execute all tutoring services, as it is designed to use different models in areas where they are better. As a result, the proposed Personalized Education Agent will be more realistic, flexible, and more relevant to the real-life process of student learning.

Model	Approx. Size	Model Type	Primary Role in the System	Key Strength	Main Limitation
Mistral-7B-Instruct-v0.3	7B	Instruction-tuned open-weight LLM	Balanced instructional baseline for general tutoring tasks	Strong balance between response quality, tuning feasibility, and practical runtime	Less specialized than Qwen for coding and less reasoning-focused than DeepSeek
Llama-3.2-3B-Instruct	3B	Compact instruction-tuned LLM	Efficient pedagogical support for short concept explanations and lightweight queries	Lower resource usage and faster inference for simpler tutoring tasks	Reduced reasoning depth on complex multi-step prompts
Llama-3.1-8B-Instruct	8B	Larger instruction-tuned LLM	Strong pedagogical model for rich concept explanation and deeper tutoring responses	Better explanation quality, stronger reasoning flow, and improved educational output	Higher latency and greater compute demand than smaller variants
Qwen 3.5 4B	4B	Technical/code-oriented instruction-tuned LLM	Efficient coding support for moderate programming and debugging tasks	Good balance between technical capability and runtime efficiency	Less detailed than the 9B variant on difficult code-heavy prompts
Qwen 3.5 9B	9B	Larger technical/code-oriented	Primary model for advanced code tutoring, debugging, and	Strongest coding assistance and richer technical explanation	Higher memory and runtime cost

		instruction-tuned LLM	implementation guidance		
Gemma-2-2B-it	2B	Compact instruction-tuned LLM	Lightweight educational support for quick clarifications and low-cost inference	Very efficient and deployment-friendly	Limited depth on advanced concept and reasoning tasks
Gemma-2-9B-it	9B	Larger instruction-tuned LLM	Stronger Gemma variant for richer tutoring output and improved reasoning	Better response detail and stronger educational quality than the 2B variant	Slower and more resource-intensive than compact models
DeepSeek-R1 7B Distilled	7B	Reasoning-focused distilled LLM	Advanced reasoning pathway for technically difficult and multi-step educational prompts	Strong logical decomposition and structured reasoning on hard prompts	Not the most efficient choice for routine or simple queries

4.2 Model Supports

The selected models were supported by a development environment designed for data preparation, parameter-efficient fine-tuning, evaluation, and inference testing. Since the Personalized Education Agent is based on multiple open-weight LLMs, the support layer had to be flexible enough to handle different model sizes while remaining practical within academic compute limits. For this reason, the project used a modular setup that combines GPU-based execution, quantized model loading, adapter-based tuning, structured preprocessing, and benchmark evaluation.

The support strategy was built around two goals. The first was to make experimentation feasible across multiple models without requiring full retraining. The second was to ensure that the final system design remained realistic for deployment, testing, and future extension. Instead of treating model support as only a training setup, the project treated it as a complete technical foundation for the tutoring platform.

4.2.1 Development and Execution Environment

The core development work was carried out within a Python notebook environment with GPU support, serving as the primary workspace for model loading, tokenizer management, prompt formatting, fine-tuning execution, inference testing, and result collection. GPU acceleration was not merely a convenience but a practical necessity, given the range of transformer models involved, which spanned compact variants through to larger, reasoning-focused architectures.

The environment additionally supported reduced-precision loading, which proved particularly valuable when working with the 7B, 8B, and 9B parameter models. Without memory-efficient execution, running fine-tuning and benchmarking workflows on these larger models would have presented considerable challenges within the resource constraints of the project. The development setup was therefore not only functional but deliberately aligned with the practical demands of the work, ensuring that experimentation and evaluation could be conducted efficiently throughout.

Table 4.2: Development Environment and Toolchain

Component	Tool / Environment	Purpose in the Project
Programming Language	Python	Core language used for preprocessing, model experimentation, fine-tuning, evaluation, and inference workflows
Development Workspace	Jupyter Notebook / Colab-style environment	Interactive experimentation, model comparison, testing, and result tracking
Compute Support	GPU-enabled execution environment	Running large language models, fine-tuning workflows, and benchmark evaluations efficiently
Deep Learning Framework	PyTorch	Core framework for model loading, training, inference, and tensor operations
Model Access Layer	Hugging Face Transformers	Loading instruction-tuned models, tokenizers, and generation pipelines

Dataset Handling	Hugging Face Datasets / structured data processing utilities	Preparing, formatting, and managing training and evaluation datasets
Fine-Tuning Framework	PEFT	Parameter-efficient fine-tuning of large models using adapter-based methods
Adapter Method	LoRA / QLoRA	Efficient domain adaptation for tutoring-oriented model specialization
Quantization Support	BitsAndBytes	Reduced-memory model loading and efficient inference for larger models
Training Workflow Support	TRL / supervised fine-tuning utilities	Managing instruction-tuning and controlled training workflows
Evaluation Benchmarks	MMLU, ARC-Challenge, HellaSwag, Winogrande, HumanEval	Measuring reasoning, technical understanding, and coding-related model quality
Text Quality Metrics	ROUGE-L, BLEU-4, BERTScore-F1	Evaluating generated responses for textual quality and alignment
Efficiency Metrics	Inference time, latency, tokens per second, perplexity	Measuring runtime practicality and deployment-oriented performance
Prompting Layer	Structured tutoring prompt templates	Framing model inputs for concept explanation, coding help, debugging, and reasoning tasks

Artifact Storage	Local notebook storage / structured project folders	Saving adapters, outputs, benchmark results, and processed datasets
Visualization Support	Tables, charts, architecture diagrams	Presenting model comparison, evaluation outcomes, and system design clearly in the report

4.2.2 Fine-Tuning and Model Adaptation Support

Parameter-efficient fine-tuning was chosen as the main adaptation strategy for this project. Full model retraining simply was not practical, given the cost and time it would have demanded across repeated experimentation with multiple models. Adapter-based tuning offered a more workable approach, allowing the models to be tailored for educational and programming-oriented tasks without touching the underlying base model weights.

This came with a number of real benefits. Memory consumption was kept manageable, experimentation cycles were shorter, and maintaining model-specific tuned variants became far less cumbersome. There was also a longer-term advantage in terms of modularity. Individual adapters can be swapped out or reused without having to rebuild the entire model pipeline from scratch, which matters quite a bit in an educational setting where different subjects or tutoring roles may eventually need their own distinct tuning approaches.

4.2.3 Quantization and Efficient Inference Support

Quantization played an important role in the model support framework. Since the selected model set includes larger variants such as Llama-3.1-8B-Instruct, Qwen 3.5 9B, Gemma-2-9B-it, and DeepSeek-R1 7B Distilled, efficient loading was required to keep experimentation practical. Reduced-precision execution helped lower memory demand and improved the feasibility of both testing and deployment-oriented inference.

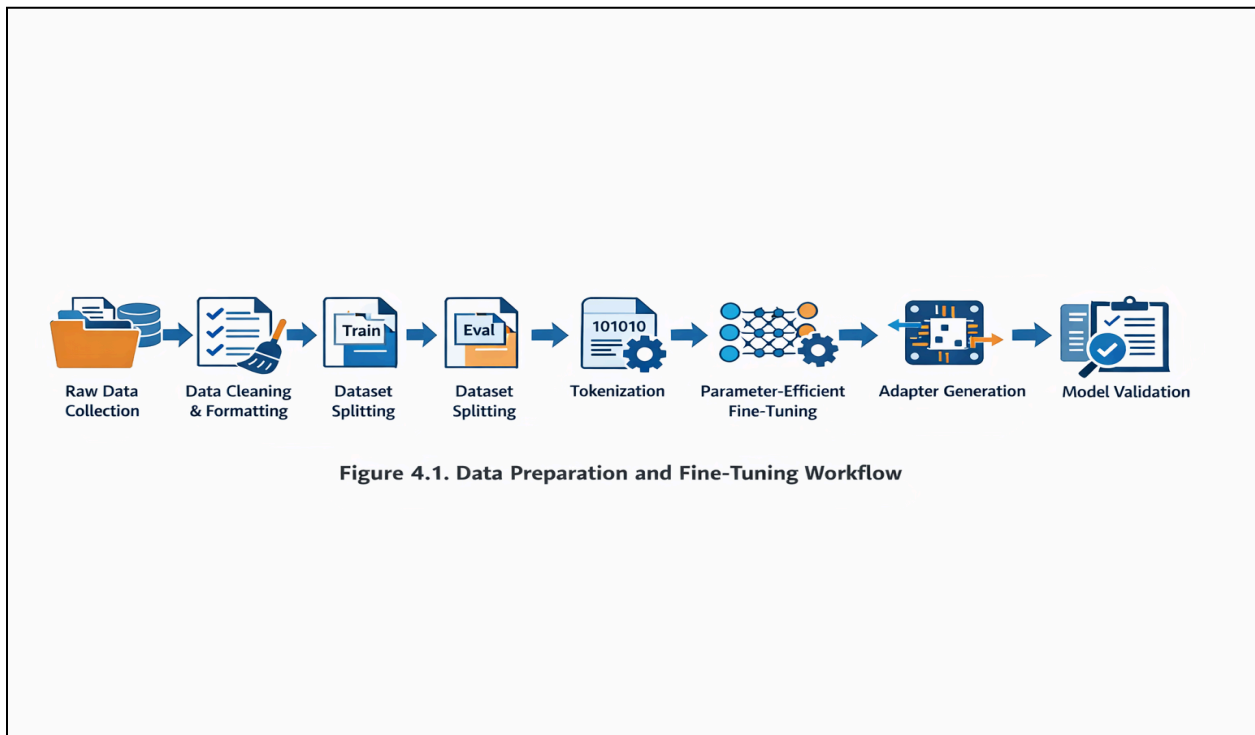
This support is important because the project is not only comparing model quality, but also considering system usability. A model that performs well but cannot be executed efficiently is less useful in a tutoring platform. Quantization, therefore, strengthened both the experimentation process and the final system design by making larger models more manageable.

4.2.4 Data Preparation Support

The model support framework also included a structured data preparation layer. Since the Personalized Education Agent is intended to produce tutoring-style responses, the raw data had to be converted into an

instruction-response format suitable for educational generation. This required cleaning, standardization, filtering, and transformation across concept explanation, programming, debugging, and evaluation tasks.

The preprocessing workflow created consistent inputs for all selected models. This was important because a fair comparison depends on using the same general data logic across the model set. It also improved output quality by ensuring that the models were exposed to better-structured educational prompts rather than noisy or inconsistent raw records.



4.2.5 Evaluation and Benchmarking Support

A separate support layer was used for model evaluation. This included benchmark execution, loss tracking, inference-time measurement, and response review. Because the project compares multiple models for different tutoring roles, the evaluation environment had to support both quantitative metrics and qualitative output analysis.

This evaluation support made it possible to compare models on reasoning, coding, efficiency, and instructional quality rather than relying on a single metric. It also strengthened the report because the final system decisions are based on measured behavior, not only theoretical assumptions. In practice, this layer connects the model development process to the later result and system design sections.

4.2.6 Integration Readiness

The model support environment was designed with future system integration in mind. Although the project focuses heavily on model development and comparison, the final goal is an intelligent tutoring

platform rather than isolated notebook experiments. For this reason, the support framework was structured so that trained models, adapters, prompt templates, and evaluation outputs can later be connected to a routing layer, an application interface, and a logging system.

This makes the selected models more than standalone research artifacts. They become system components that can be invoked for different tutoring tasks based on their strengths. That integration readiness is one of the reasons the support structure is an important part of the workbook.

Table 4.3: Frameworks and Their Functions

Framework / Tool	Function in the Project
PyTorch	Core deep learning framework used for model execution, training, and inference
Hugging Face Transformers	Loading pre-trained instruction-tuned models and tokenizers, and supporting text generation workflows
Hugging Face Datasets	Managing, formatting, and processing structured datasets for training and evaluation
PEFT	Enabling parameter-efficient fine-tuning so large models can be adapted without full retraining
LoRA	Low-rank adaptation method used to fine-tune models efficiently for tutoring-oriented tasks
QLoRA	Quantized low-rank adaptation is used to reduce memory cost while preserving fine-tuning effectiveness
BitsAndBytes	Supporting low-bit quantization for memory-efficient model loading and faster experimentation

TRL / SFT utilities	Supporting supervised fine-tuning workflows and controlled instructional training
Tokenizers	Converting raw text prompts and responses into model-readable token sequences
Benchmark evaluation scripts	Running model assessments on reasoning, coding, and educational evaluation tasks
Prompt templates	Structuring model input for concept explanation, code tutoring, debugging, and advanced reasoning
Notebook environment	Supporting interactive experimentation, comparison, training, and result analysis
Storage and artifact management	Saving processed datasets, adapters, evaluation outputs, and generated results in an organized workflow
Visualization tools	Creating tables, figures, and summary charts used to present model performance and system design

4.3 Model Comparison and Justification

Once the proposed models are chosen and the technical environment that enables them has been identified, the next step is to compare the proposed models in an organized manner and explain their purpose in the Personalized Education Agent. This is not a competition in which one should find a common winner. Rather, it aims at establishing the most appropriate models to be used in various educational activities like explaining concepts, tutoring codes, debugging, advanced reasoning, and efficient deployment. The reason behind this is that the ultimate system should be an adaptive educational platform rather than a single-model chatbot.

This comparison was made according to five pragmatic criteria. The first is the quality of instructions that is a measure of how easily and conveniently a model can explain concepts. The second is reasoning depth, which is important when dealing with many steps of technical questions and complicated problem-solving. The third is coding support that comprises code generation, debugging, and clarification of the implementation logic. The fourth is efficiency, such as response speed, memory demand, and deployment practicality. The fifth one is adaptability, i.e., the appropriateness of the model in terms of fine-tuning and incorporation into the project pipeline.

4.3.1 Comparison Basis

The models chosen were not randomly selected. Each of them is an alternative design of the tutoring system. Mistral-7B-Instruct-v0.3 is an instruction-tuned baseline that is balanced. It is possible to compare smaller and bigger models of pedagogy within the same family using Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct. The Qwen 3.5 4B and Qwen 3.5 9B enable the comparison of smaller and larger models with excellent technical and coding relevance. Another small-versus-large comparison has been offered by Gemma-2-2B-it and Gemma-2-9B-it with a high emphasis on efficiency in instruction. DeepSeek-R1 7B Distilled is the model that has the reasoning as its focus.

The advantage of this structure is that it enables the project to compare not only across family models but also within a family. Consequently, the report can not only justify what model is powerful, but it can also justify why that model is powerful in a specific position in the tutoring system.

4.3.2 Mistral-7B-Instruct-v0.3

Mistral-7B-Instruct-v0.3 is recommended as a moderate baseline model in the project. It provides a reasonable balance between the instruction-following capability, small model size, and reasonable tuning possibilities. It has high-quality output and enhanced reasoning as compared to smaller models. It is also more efficient and easier to use with limited computing resources than larger models.

Its main strength is balance. It can produce systematic education responses, yet it is not so great that it would be inconvenient to experiment. This renders it a powerful reference point during the establishment of the model. It has its primary drawback because it is not as specialized in programming as Qwen and as reasoning-oriented as DeepSeek. That is why Mistral is the most suitable in terms of being a trusted general instructional model and a powerful experimental baseline.

4.3.3 Llama-3.2-3B-Instruct and Llama-3.1-8B-Instruct

To compare the impact of the scale within the same architectural family, Llama models were chosen. Llama-3.2-3B-Instruct is the smaller and more performant one, whereas Llama-3.1-8B-Instruct is the stronger one with more profound reasoning and more educational output. The significance of this comparison is that the tutoring system must have the lightweight support of simple interactions, but with more solid support of the heavy load in the form of an explanation.

Llama-3.2-3B-Instruct is suitable because it is a cheaper model in the case of shorter conceptual response and efficient deployment. It is practical when the system requires clear yet rather light tutoring support.

The reason why Llama-3.1-8B-Instruct should be chosen is that it is a more effective pedagogical model, since it is more appropriate in explaining complex concepts, multi-step instructions, and more detailed tutoring behavior.

The analogy of the two models assists in creating a distinct size-versus-quality trade-off. The 3B model enhances practicality and speed, whereas the 8B model enhances depth and richness of instruction. They collectively help to make the project stronger because they demonstrate that both efficient and more capable pedagogical models can be used on the final architecture.

4.3.4 Qwen 3.5 4B and Qwen 3.5 9B

The Qwen models have been incorporated due to the high level of relevance to programming-oriented learning tasks. This line of the model is particularly crucial since the Personalized Education Agent is likely to facilitate the clarification of codes and debugging, as well as assist the algorithm. The smaller and more efficient technical model is Qwen 3.5 4B, and the stronger version is Qwen 3.5 9B, used in the case of more demanding coding and implementation functions.

Qwen 3.5 4B is rationalized because it is an effective model to moderate support programming and reduce resource experimentation. It comes in handy when the system requires a quick code-oriented interaction, but at a lower cost than a larger model. The reason why Qwen 3.5 9B is justifiable as the better programming model is that it can offer better debugging capabilities, deeper code descriptions, and stronger technical output on more challenging prompts.

This comparison is among the most crucial in the report as it directly underlines the objectives of coding and debugging of the tutoring system. The 4B model is useful in cases where efficiency is important, whereas the 9B model is used in cases of technically challenging educational activities. Collectively, they act as a solid reason why a code-based model line should be incorporated into the ultimate system.

4.3.5 Gemma-2-2B-it and Gemma-2-9B-it

The Gemma models were chosen in order to compare compact models with larger instruction-tuned models of the same model. Gemma-2-2B-it is the most efficient variant, whereas Gemma-2-9B-it is the more capable version and stronger. Such a comparison is not a bad idea, as it is possible to assess the extent to which educational value may be gained with a lightweight model and improve it when a bigger one is employed.

Gemma-2-2B-it is also justified because it is a deployment-friendly model used for fast clarification, brief conceptual answers, and basic education interaction. It is small and thus appealing to support low latency and experiment with it practically. The use of gemma-2-9B-it is justified since it has stronger reasoning, detailed explanations, and a structure of responses to more complex prompts.

Another critical design lesson of the project, which is supported by the Gemma comparison, is that not all queries require the largest model. Even in the context of numerous interactions with students, a small and effective model can be of some service. Simultaneously, the 9B version indicates that we can perform better when there is a more demanding task.

4.3.6 DeepSeek-R1 7B Distilled

The reasoning model chosen in the lineup was DeepSeek-R1 7B Distilled. It primarily assists with technically challenging and multi-phase educational prompts in which logical structure and greater analysis should be significant. This renders it particularly useful when it comes to explanations of more complex concepts, breaking down problems, and technical issues that require more critical thinking to resolve.

DeepSeek is more closely associated with advanced reasoning as compared to the more balanced or deployment-oriented models. Its greatest advantage is that it is the most appropriate when it comes to prompts that demand a causal sequence of explanation instead of a brief direct answer. This makes it have a distinct niche in the tutoring system as the higher-order reasoning route.

Its major disadvantage is that it is not the most efficient when it comes to all forms of interaction. Due to its specialization in more in-depth thinking, it has a stronger application in complex prompts as opposed to simple educational questions. The justification of this becomes very simple: DeepSeek is not added as a general model, but rather the model that will be used in hard, reasoning-heavy tutoring activities.



Fig : Model comparison on different performance data

4.3.7 Cross-Model Comparison

Comparing the selected models to each other, each of them brings new value to a system. Mistral-7B-Instruct-v0.3 has a good, balanced base. The Llama models enhance pedagogical explanation

and demonstrate the scale effect of models on the quality of tutoring. The most applicable models are the Qwen models that are used to provide programming and debugging. Gemma models show effective interaction in education between the smaller and larger variants of instruction tuned. DeepSeek-R1 7B Distilled includes robust support of reasoning behind the most challenging technical queries.

This comparison demonstrates that there is no ideal model that is the best for all needs. Some models are faster, models that are more instructionally rich, models that are more coded, and models that are more advanced in their reasoning. This is the reason why a single-model approach to strategy is not to be used in the end system. The actual needs of an adaptive tutoring platform are most in line with a multi-model design.

Table 4.4: Cross-Model Comparison and Justification

Model	Instructional Quality	Reasoning Depth	Coding Support	Efficiency	Best Use Case	Justification
Mistral-7B-Instruct-v0.3	High	Moderate-High	Moderate	Moderate	General tutoring	Balanced baseline model
Llama-3.2-3B-Instruct	Moderate	Moderate	Low-Moderate	High	Short concept help	Efficient pedagogical model
Llama-3.1-8B-Instruct	High	High	Moderate	Moderate	Deep concept explanation	Stronger tutoring quality
Qwen 3.5 4B	Moderate	Moderate	High	High	Efficient coding helps	Compact technical model
Qwen 3.5 9B	High	High	Very High	Moderate-Low	Advanced coding support	Strongest code-focused model

Gemma-2-2B-it	Moderate	Low-Mode rate	Low	Very High	Quick clarifications	Lightweight support model
Gemma-2-9B-it	High	High	Moderate	Moderate-Low	Richer tutoring output	Stronger Gemma variant
DeepSeek-R1 7B Distilled	High	Very High	Moderate	Moderate	Advanced reasoning tasks	Reasoning-specialized model

4.3.8 Final Justification for the System

The best conclusion that can be drawn through the comparison is that the Personalized Education Agent ought to be created as a role-based multi-model system. Mistral-7B-Instruct-v0.3 is balanced in its teaching point. The Llama-3.2-3B-Instruct and Gemma-2-2B-it assist in efficient low-cost learning communication. Pedagogical quality is supported better with Llama-3.1-8B-Instruct and Gemma-2-9B-it. Qwen 3.5 4B and Qwen 3.5 9B enhance the debugging and coding aspect of the system. DeepSeek-R1 7B Distilled allows more reasoning if a deeper explanation is necessary.

This is better than choosing the best model due to the fact that students do not ask a single type of question. Some require immediate elucidation, others require assistance with codes, and those that require in-depth support of reasoning. The last architecture must then apply each model in the area where it works best. This renders the system more adaptable, more realistic, and much more consistent with the objectives of an individualized educational support.

4.4 Model Evaluation Methods

After comparing and justifying the selected models, the next step is to define how they were evaluated. Since the Personalized Education Agent is designed as a tutoring system rather than a general text generation project, the evaluation process had to measure more than raw model accuracy. A useful model in this system must be able to explain concepts clearly, support coding tasks, respond with reasonable efficiency, and remain practical for integration into a multi-model educational platform. For this reason, the evaluation framework combined benchmark-based testing, fine-tuning indicators, runtime analysis, and qualitative educational review.

This section would assist in giving an explanation of how the selection of the models was done in a consistent and meaningful manner. The evaluation techniques were aimed at capturing both technical and educational value. This is important since a model that performs well in a standard benchmark may remain ineffective as a tutor in the event that it produces answers that are vague, poorly structured, or even too concise. On the same note, when the inference cost of a model is prohibitive, it might be difficult to apply in practice even when it provides good responses. Consequently, the end evaluation design reflects well the objectives of the project.

4.4.1 Evaluation Framework

The overall evaluation framework was divided into four parts. The first part focused on benchmark-based capability testing. This was used to measure reasoning, knowledge performance, and technical task handling across the selected models. The second part focused on fine-tuning effectiveness, including how well the models adapted to educational and coding-oriented data. The third part focused on runtime efficiency, including latency, inference time, and throughput. The fourth part focused on qualitative output review, where the educational usefulness of the model responses was assessed directly.

This four-part structure was chosen because the project compares several models that serve different purposes in the final system. Smaller models may perform better in speed and deployment practicality, while larger models may perform better in reasoning and richer explanation. By using multiple evaluation layers, the project avoids reducing all model decisions to a single score and instead evaluates each model in terms of its actual contribution to the tutoring system.

4.4.2 Benchmark-Based Evaluation

The evaluation of the overall capability of the chosen models, on the reasoning, technical understanding, and structured response tasks, was carried out through benchmark-based evaluation. These standards facilitate the comparison of the models in the conditions of the general testing and offer a more objective analysis ground. The benchmark set was selected based on the needs related to the explanation of the concept, coding support, and technical learning assistance, as the project is aimed at those aspects.

The benchmark group chosen will be MMLU, MMLU-CS, or MMLU-STEM, where applicable, ARC-Challenge, HellaSwag, and Winogrande. The selection of these benchmarks is due to the fact that they cover throughout the reasoning, subject knowledge, commonsense knowledge, and technical task interpretation. Models that pass these tests are usually more favorable to facilitate academic tutoring work, particularly when accompanied by teaching fine-tuning.

The benchmark results were not regarded as the sole criteria of selecting the model. They were, instead, components of the general evaluation strategy. This is significant in that the quality of educational tutoring is also determined by clarity, structure, and suitability to the needs of students, which cannot be entirely captured using standard benchmarks.

Table 4.5: Evaluation Metrics and Their Purpose

Evaluation Metric	Purpose in the Project
MMLU	Measures broad academic and reasoning performance across multiple subjects
MMLU-CS / MMLU-STEM	Evaluates technical understanding in computer science and STEM-focused tasks
ARC-Challenge	Tests reasoning ability on more difficult question-answer tasks
HellaSwag	Measures commonsense reasoning and contextual understanding
Winogrande	Evaluates sentence-level reasoning and contextual decision making
Training Loss	Tracks learning progress during fine-tuning and shows model adaptation quality
Perplexity	Measures how well the model predicts text and reflects language modeling quality
Inference Time	Shows how long the model takes to generate a response
Tokens per Second	Measures generation speed and runtime efficiency
P50 Latency	Captures median response delay for practical system performance evaluation

ROUGE-L	Evaluates overlap between generated output and reference responses
BLEU-4	Measures n-gram similarity, especially useful for generated text comparison
BERTScore-F1	Evaluates semantic similarity between generated and reference outputs
HumanEval Pass@1	Measures coding correctness by checking whether the generated code passes evaluation in one attempt
Qualitative Pedagogical Review	Assesses clarity, explanation quality, structure, and educational usefulness of responses

4.4.3 Fine-Tuning Performance Evaluation

Since the project relies on parameter-efficient fine-tuning rather than full retraining, it was important to evaluate how well the selected models adapted to the tutoring domain. Fine-tuning performance evaluation focused on training loss, checkpoint stability, and visible improvement in response structure after adaptation. The goal was to determine whether the models became more aligned with educational prompting and tutoring-style output.

Training loss was used as the primary numerical indicator during adaptation. A stable downward trend suggested that the model was learning the instructional format and becoming better aligned with the target data. In addition to loss, the project also considered whether the outputs became more structured, more explanation-oriented, and more useful for tutoring after fine-tuning. This was especially important because a lower loss value alone does not guarantee better educational responses.

For Mistral in particular, the evaluation also considered metrics such as ROUGE-L, BLEU-4, BERTScore-F1, HumanEval Pass@1, and P50 latency. These metrics were useful because they captured both textual similarity and practical coding-related performance. For Gemma, additional indicators such as perplexity, inference time, and tokens per second were also relevant for judging practical performance after tuning and inference testing.

4.4.4 Runtime and Efficiency Evaluation

Since the Personalized Education Agent is intended to support interactive educational use, runtime performance was treated as a major evaluation category. The project did not assume that the best model is always the largest or most capable one. Instead, the models were evaluated on whether they provide useful performance at a practical cost. This is especially important because the final system is expected to serve different kinds of queries with different complexity levels.

Runtime evaluation included inference time, latency, and throughput. Inference time measures how long the model takes to generate a response. Latency is important because slow responses can reduce the usefulness of the system in real tutoring scenarios. Throughput, measured using tokens per second where available, helps evaluate how efficiently the model can handle generation.

This part of the evaluation is essential because it directly supports the final multi-model system design. If a smaller model can provide acceptable tutoring support much faster, it may be better suited for lightweight educational queries. If a larger model produces much better reasoning quality, it may be reserved for more complex prompts. The runtime evaluation, therefore, helps justify model routing in the final architecture.

4.4.5 Qualitative Pedagogical Review

The important segment of the evaluation model involved the practical usefulness of the model outputs within a school setting. This qualitative review was required since benchmark scores and training measures do not give an accurate picture of the effectiveness of a tutor as a model. An answer can be technically right but still fail to pass the educational test because the answer is not specific enough or sometimes too brief, too complicated, or too badly organized.

The qualitative review involved a review of the models based on the ability to provide clear explanations, coherent step by step logics, relevant examples, helpful debugging tips, and easy-to-understand code support. This section of the assessment was particularly significant to the comparison of pedagogical models like Llama, coding-oriented models like Qwen, and reasoning-oriented models like DeepSeek. The review also assisted in determining whether smaller models could still be useful in offering some form of educational value to shorter tasks.

The review was conducted based on representative prompts of the target domains of the project, such as concept explanation, programming support, debugging, and advanced reasoning questions. The outputs were rated then in terms of clarity, coherence, usefulness, and how in tune the tutoring objective was. This qualitative layer rendered the assessment much more pertinent to the real purpose of the system.

4.4.6 Error Analysis

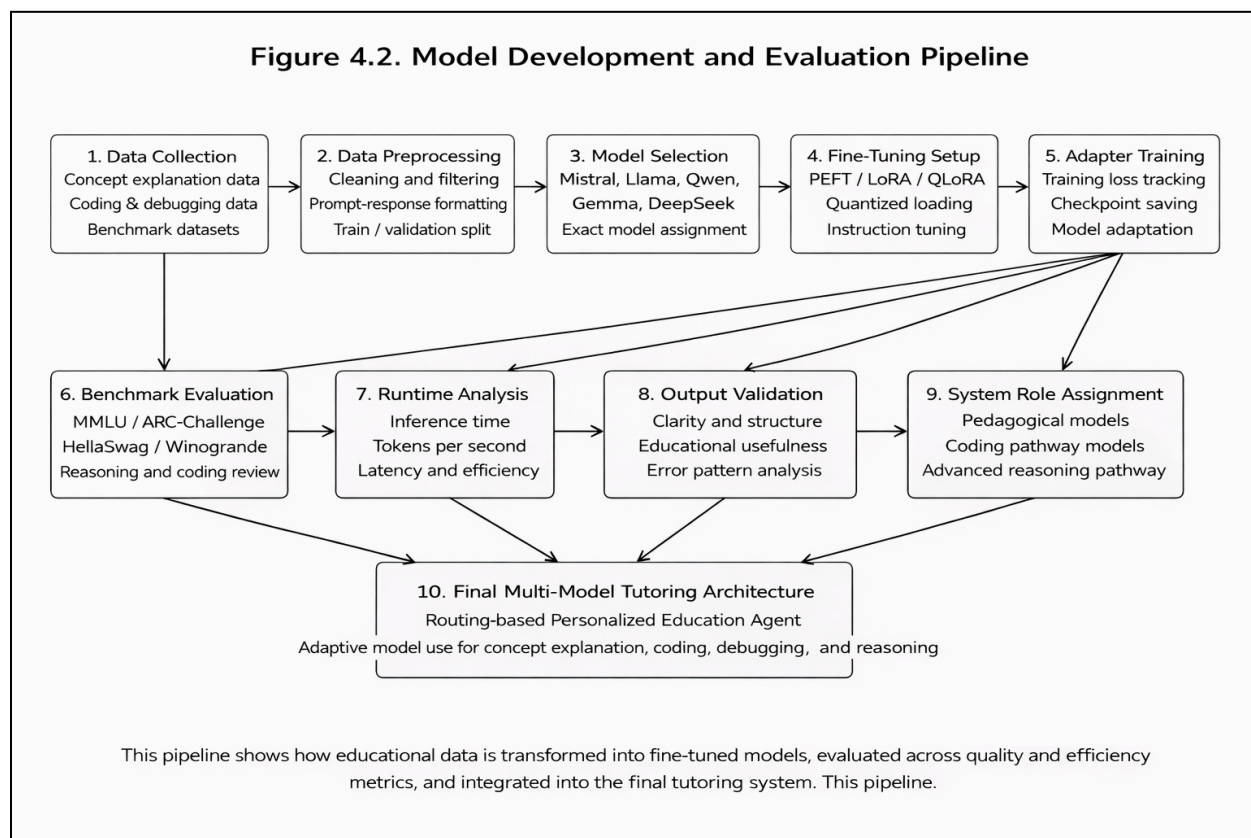
Error analysis was also included as part of the evaluation process. The purpose of this step was to understand how and where the models fail, not just where they perform well. This is important because system design decisions are often shaped by limitations as much as by strengths. A model that produces high-quality answers but fails on speed may need a limited role. A model that is fast but too shallow may still be useful for short clarifications.

The project examined common failure patterns such as incomplete reasoning, overly short explanations, unnecessarily verbose responses, weak debugging support, or technically correct but poorly explained outputs. This analysis helped clarify which models are best suited for which tutoring roles. It also helped show that the final architecture should not try to force one model to handle every type of educational interaction.

4.4.7 Multi-Model Suitability Evaluation

The last aspect of the assessment was role-based suitability in the system. The project is planned as a multi-model platform, so it was necessary that the performance of every model alone should be observed, but also should work effectively within the tutoring architecture. This involves the role that the model will play exclusively, as well as how the model will act in complement with other models, and the strengths that the model offers in its position in the system.

An example is that a smaller model might not excel in the reasoning score, but still be of great value, assuming that it assists in low-latency tutoring. The code-based model can not be the best general explainer, but it can still be necessary when it comes to debugging and programming. A model based on reasoning can prove too costly to answer each query, but can be very useful on the most challenging prompts. This is among the most vital sections of the evaluation process since this perspective of the system underpins the ultimate architecture of the Personalized Education Agent.



4.5 Model Validation and Evaluation Results

Once the evaluation framework was formed, the next step was to validate the models chosen and explain the results within the framework of the goals of the Personalized Education Agent. This section will be used to record the performance of the models on grounds of reasoning, coding support, efficiency, and educational usefulness, and explain how the findings were used to inform the design of the final system. Since the multi-model tutoring architecture was used in the project, validation was not meant to single out one and only universally effective model, but to establish how different models are most effective in specific instructional functions.

The results of the validation showed a different pattern between the models that have been selected. Models with greater capability and size always performed better on concept-heavy, reasoning-intensive, and task-based explanations. Smaller models, on the other hand, performed better in terms of efficiency, low latency, and lightweight deployment. Models that were coding-oriented exhibited an increased ability in debugging and support of implementation. The results support the initial design choice of the project, Personalized Education Agent, which is to use various models depending on the specific tutoring requirements instead of applying a single model to all the work.

4.5.1 Benchmark Validation Results

The benchmark scores showed that bigger models of each model family tend to perform better on reasoning and knowledge-based tests compared to smaller models. Llama-3.1-8B-Instruct was higher on explanation-rich and multi-step items than Llama-3.2-3B-Instruct, and there was more depth and a higher quality of instructional response. Qwen 3.5 9B was better than Qwen 3.5 4B on technical prompts, especially where something needed deeper coding logic or more complex implementation assistance. Gemma-2-9B-it also exhibited better performance as compared to Gemma-2-2B-it in reasoning-oriented tests.

DeepSeek-R1 7B Distilled performed highly on structured reasoning problems and appeared to be one of the most useful models on technically challenging prompts. Mistral- 7B-Instruct-v 0.3 was a high-quality, balanced baseline, providing consistent results with a wide variety of task types, although not always leading the expert fields. On the whole, benchmark validation proved that both model scale and specialization are important in performing on educational tasks.

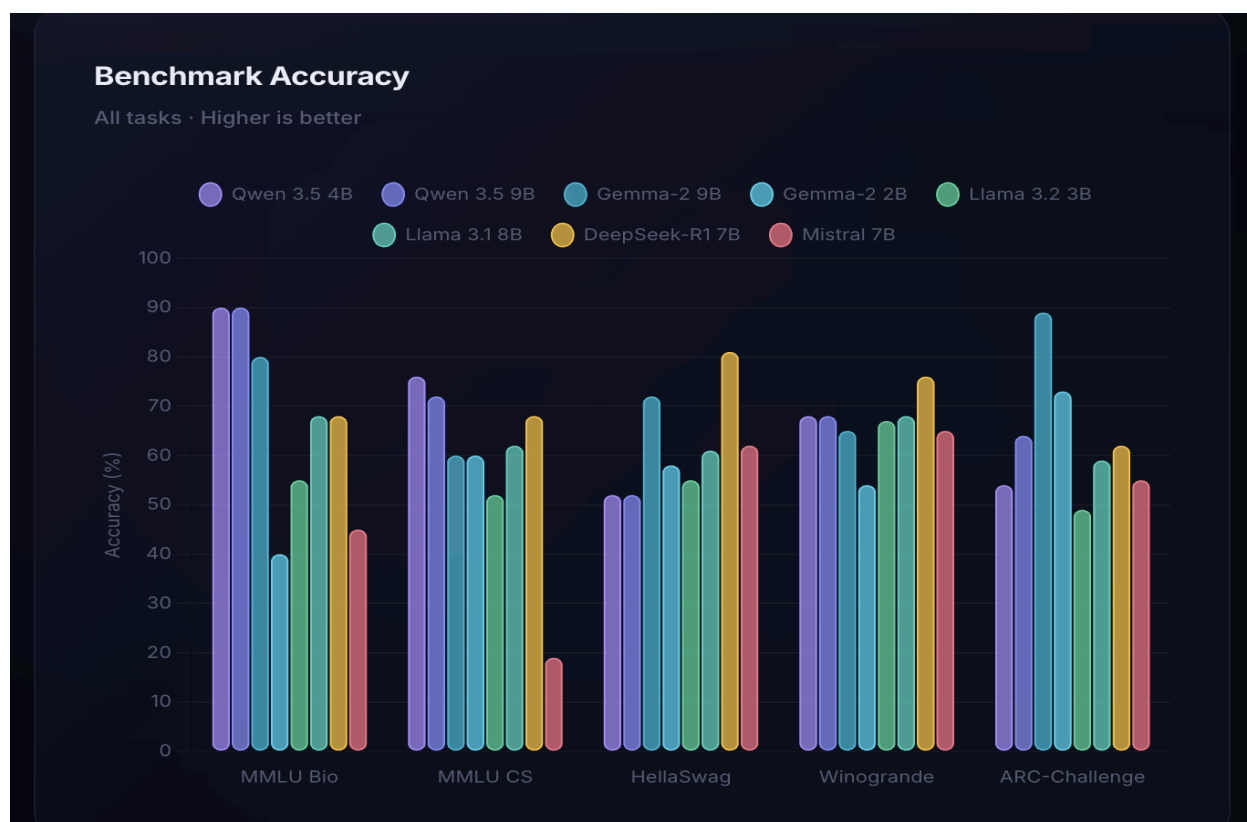


Fig : Benchmark accuracy results

4.5.2 Fine-Tuning Validation Results

The results of the fine-tuning experiments showed that parameter-efficient adaptation revealed the educational usefulness of the models to a significant extent. The responses obtained with subsequent tuning were much closer to tutoring-style output, especially in regard to the structure of the explanation, clarity, and task relevance. This result can be regarded as a milestone of the project, as it indicates that the chosen open-weight models can be successfully transformed to be used in pedagogy without having to train them completely.

In the case of Mistral-7B-Instruct-v 0.3, the fine-tuning procedure provided stable training dynamics and quantifiable training scores in text-based and coding-based assessment metrics. The modified model had a better structure of instruction and increased correspondence to the tutoring goal. With Gemma, the evaluation process also indicated that the larger variant generated more robust products, whereas the smaller variant was not useless, as it was faster and lighter to interact in education. These results confirm the applicability of the tuning of adapters as a feasible development process of the ultimate system.

4.5.3 Runtime and Efficiency Results

Measuring the behavior of the models at run time showed that there was a clear inverse correlation between computational cost and model performance. Lighter versions, such as Llama-3.2-3B-Instruct,

Qwen 3.5 4B, and Gemma-2-2B-it, had reduced resource usage and produced fast inference times. This type of model is especially useful with tutoring applications where a quick explanation is more important than comprehensive pedagogical breadth. Therefore, their functional features make them quite appropriate for interactions with low overhead and low cost conditions.

Larger versions, however, namely Llama 3.1 8B Instruct, Qwen 3.5 9B, and Gemma 2 9B, required more computational power but provided more detailed and pedagogically useful answers. DeepSeek-R1 7B Distilled and Mistral-7B-Instruct-v0.3, which occupied an intermediate-strong category, were able to afford much reasoning power and instructional value and were more practical in practice than some of the largest systems. These results highlight that there is a need to match model dimensionality with the task exigency instead of mindlessly applying the same architecture to all interaction modalities.

Table 4.6: Validation and Result Summary

Model	Validation Outcome	Main Strength Observed	Main Limitation Observed	Final Role in the System
Mistral-7B-Instruct-v0.3	Strong overall performance	Balanced instructional quality and stable fine-tuning behavior	Less specialized than Qwen for coding and DeepSeek for reasoning	General instructional baseline
Llama-3.2-3B-Instruct	Good lightweight performance	Efficient concept support with lower runtime cost	Limited depth on complex reasoning tasks	Lightweight pedagogical support
Llama-3.1-8B-Instruct	Strong pedagogical performance	Richer explanation quality and stronger multi-step reasoning	Higher latency than smaller variants	Main concept explanation pathway
Qwen 3.5 4B	Good technical performance	Efficient coding and debugging support	Less detailed than the 9B version	Efficient coding support

Qwen 3.5 9B	Very strong technical performance	Best coding, debugging, and implementation guidance	Higher compute and runtime cost	Advanced coding pathway
Gemma-2-2B-it	Useful low-cost performance	Fast response for short educational interactions	Weaker reasoning on advanced prompts	Quick clarification support
Gemma-2-9B-it	Strong larger-model performance	Better explanation depth than the 2B version	Less efficient than compact models	Richer tutoring support
DeepSeek-R1 7B Distilled	Strong reasoning-focused performance	Excellent structured reasoning on difficult prompts	Not ideal for routine low-complexity queries	Advanced reasoning pathway

4.5.4 Coding and Debugging Performance

The validation results showed that the Qwen models were especially strong in code-related tasks. Qwen 3.5 9B delivered the strongest programming support among the compared models in code explanation, implementation guidance, and debugging-oriented outputs. Qwen 3.5 4B also performed well, especially when efficiency was important, but the larger version produced more detailed and reliable technical responses on difficult prompts.

Mistral-7B-Instruct-v0.3 also showed meaningful performance in coding-related evaluation, especially because its fine-tuning workflow included technical and coding-oriented metrics. It served as a useful, balanced model for educational programming support, though it remained less specialized than the strongest Qwen variant. These results confirm that coding assistance should be treated as a dedicated function in the tutoring system and that the final architecture should reserve a clear role for the Qwen line.

4.5.5 Pedagogical Response Validation

Qualitative evaluation of the products showed that high benchmark scores alone could not be used to adequately define educational utility. Correctness combined with a high quality of explanation, logical flow, and a format that was friendly to a learner were the most successful types of tutoring responses. In

this regard, Llama-3.1-8B-Instruct was shown to be better at explaining concepts and producing more detailed tutoring responses, always producing more detailed educational output compared to the smaller Llama.

DeepSeek-R1-7B Distilled was also performing well in the pedagogical validation tests when deeper reasoning was required due to prompting. Its replies were particularly useful to high technical descriptions, but there were some cases where very severe length control was needed to avoid the vagueness of undue wordiness. Gemma-2-9B-it and Mistral-7B-Instruct-v0.3 also produced good educational materials, especially when the prompts needed a moderate description, and not a very specific argument. These findings highlight the fact that the pedagogical utility is not only determined by the ability of the model, but also the fit of the model to the tutoring role.

4.5.6 Small-versus-Large Model Findings

The validation procedure produced a strong distinction between smaller and bigger models as forming the same architectural lineage. The comparative studies carried out on the Llama, Qwen, and Gemma families showed that the larger versions all exhibited better reasoning abilities, provided more detailed and thorough responses, and had more extensive tutoring-support functionalities. The smaller variants, on the other hand, were able to maintain usefulness due to less computational load, shorter inference time, and were applicable on cost limited deployments.

There are consequential implications for the structure of the future tutoring platform as a result of this observation. It eliminates the assumption of using the most powerful model in all instructional activities. In practice, a lightweight model is capable of providing sufficient pedagogical support with significantly greater efficacy in many situations. Based on it, the validation results support the idea of role-based system architecture where less resource-intensive models handle routine and less complex inquiries, and the more demanding learning prompts are directed to high-capacity variants.

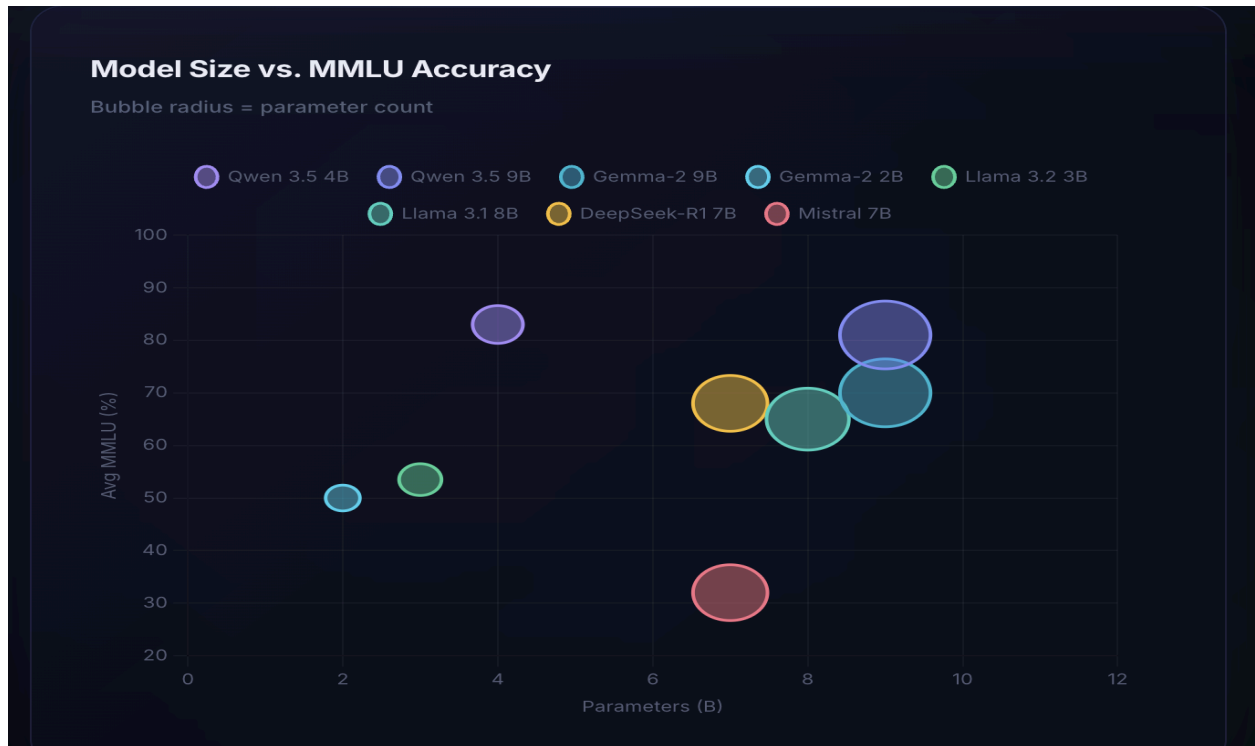


Fig : Model size vs system MMLU accuracy

4.5.7 Final Role Assignment Based on Validation

The outcomes of the final validation support a clear distribution of roles of the chosen models. Mistral-7B-Instruct-v0.3 is a baseline of balanced instructions, as well as a strong reference point for fine-tuning. Llama-3.2-3B-Instruct and Gemma-2-2B-it are effective auxiliary models regarding lighter interactions in learning. Llama-3.1-8B-Instruct and Gemma-2-9B-it are stronger tutoring models that provide more detailed explanations and are able to think more profoundly. Qwen 3.5 4B and Qwen 3.5 9B are coded to the coding and debugging pathway, the 9B being the technical model perceived to be niche. DeepSeek-R1 7B Distilled is the state-of-the-art reasoning model of technically challenging prompts.

This role assignment is the most interesting result of the validation process; it proves the transition of the generic model comparison to the system-level design. The results are beyond the performance indicators, and they explain how and why each model plays a significant role in the fulfilment of the Personalised Education Agent.

Table 4.7: Runtime and Efficiency Comparison

Model	Runtime Efficiency	Response Speed	Resource Demand	Efficiency Summary	Best Operational Use

Mistral-7B-Instruct-v0.3	Moderate	Moderate	Moderate	Balanced runtime and quality	General-purpose tutoring
Llama-3.2-3B-Instruct	High	Fast	Low	Efficient and lightweight	Short concept support
Llama-3.1-8B-Instruct	Moderate-Low	Slower	High	Better quality with higher cost	Deep concept explanation
Qwen 3.5 4B	High	Fast	Moderate	Efficient technical support	Fast coding assistance
Qwen 3.5 9B	Moderate-Low	Slower	High	Stronger coding quality with more compute cost	Advanced coding and debugging
Gemma-2-2B-it	Very High	Very Fast	Low	Best low-cost efficiency	Quick clarifications
Gemma-2-9B-it	Moderate-Low	Slower	High	Better tutoring depth, but less efficient	Richer educational output
DeepSeek-R1 7B Distilled	Moderate	Moderate-Slow	Moderate-High	Reasoning strength with moderate efficiency trade-off	Complex reasoning tasks

4.5.8 Section Summary

The overall results of the validation confirm that the chosen models offer different benefits to the tutoring system. Larger-scale models enhance the quality of reasoning, depth of explanations, and technical support, but smaller-scale models are mainly faster and more efficient. Specifically, Qwen adds to the coding pathway, Llama adds to the explanatory power of pedagogy, Gemma adds to the ability to interact efficiently and in large-scale, Mistral adds to the strength of a balanced baseline, and DeepSeek adds to the ability to reason.

These results provide strong arguments in favor of the implementation of a multi-model architecture of the Personalized Education Agent. The next section is a transition between model-level validation and system-level design to outline the needs that should be fulfilled by the final tutoring platform.

5.1 System Requirements Analysis

After validating the selected models, the next step is to define the system requirements for the Personalized Education Agent. This section translates the model-level findings into a complete system perspective by identifying what the platform must do, who will use it, and what technical and functional capabilities it must support. The goal is to ensure that the project is presented not only as a model comparison study, but as the design of a practical intelligent tutoring system.

The Personalized Education Agent is intended to support concept explanation, code tutoring, debugging help, and structured reasoning for technical questions. Because of this, the system must go beyond simple question-answer behavior. It must accept different types of academic queries, identify the nature of the request, select an appropriate model pathway, generate a useful response, and maintain a structure that supports future personalization and analytics.

5.1.1 System Purpose

The primary aim of the system is to offer interactive and adaptive academic assistance to the students. The platform will be created to assist learners to learn concepts, solving technical issues, enhancing their ability to code, and getting step-by-step instructions whenever they encounter tough questions. The system is supposed to act like an educational assistant, and not a generic conversation, although it is not based on a general chatbot.

The second goal of the system is to provide support for various tutoring styles on a single platform. Certain student questions should have only a brief clarification, and others need either a code-level explanation or a more profound argument. The system should therefore be able to vary depending on the type of request. It is among the most important reasons why the project is designed in the form of a multi-model.

5.1.2 Primary Users

Students are the main consumers of the system. The interaction between students and the platform takes the form of conceptual questions, needs assistance with code, prompts involving debugging, or prompts

requesting clarifications of technical information. The system should hence offer answers that are precise, pertinent, and learning beneficial.

The second user group consists of instructors or reviewers who can make subsequent use of the system to view model behavior, see the quality of interaction, or assess the utility of the tutoring response. This generates the necessity of logging, transparency, and an orderly system design that will be reviewed and extended. The third category of users comprises developers or system administrators who take care of updates to the models, trigger templates, routing logic, and evaluation processes.

5.1.3 Functional Requirements

The system must support a set of core functions that align with the goals of the project. The first functional requirement is query intake. The platform must accept student prompts in natural language and support different categories of educational requests. The second requirement is task identification. The system must be able to distinguish between conceptual explanation, coding help, debugging support, and more advanced reasoning tasks.

The third requirement is model routing. Once the query type is understood, the system must select the model or model group best suited for that task. The fourth requirement is structured response generation. The selected model must return a response that is educationally meaningful and appropriate for the user's request. The fifth requirement is response delivery, where the output is shown in a readable and learner-friendly format.

The system must also support logging of queries, selected models, and output behavior. This is necessary for performance review, future analytics, and system improvement. In addition, the platform should support future extension into learner-aware personalization, where past interactions or mastery information can influence the tutoring response.

Table 5.1: System Users and Functional Requirements

User / Stakeholder	Primary Need	Functional Requirement
Student	Understand concepts and solve technical questions	Submit queries and receive clear, structured tutoring responses
Student	Get help with programming tasks	Receive code explanation, implementation guidance, and debugging support

Student	Get support for difficult multi-step questions	Access stronger reasoning-oriented response pathways when needed
Instructor / Reviewer	Observe tutoring quality and system usefulness	Review outputs, monitor tutoring behavior, and assess educational relevance
Developer / System Administrator	Maintain and improve the platform	Update models, adapters, prompts, routing rules, and evaluation workflows
System	Handle different types of academic queries	Identify the task type and route each query to the most suitable model
System	Deliver useful educational responses	Generate structured outputs for explanation, coding, debugging, and reasoning tasks
System	Support evaluation and future analytics	Store logs, outputs, and performance data for monitoring and improvement

5.1.4 Non-Functional Requirements

Besides more fundamental features, the system should meet a number of non-functional requirements. The former is the quality of response. The system should generate clean and organized, as well as technically stable, e-teaching products. The second is efficiency. The site has to answer within a realistic time frame, more so when making straightforward queries; the quick service will be essential.

Scalability is the third requirement. It should be designed in such a way that it can be easily extended to include additional datasets, additional subjects, additional model variants, and additional users without a full redesign. The fourth requirement is maintainability. The platform should be designed in such a manner that models, adapters, prompts, and routing rules can be updated separately. The fifth requirement is reliability, i.e, the system must generate fairly stable and consistent behavior when used repeatedly.

5.1.5 Educational Intelligence Requirements

Since this project is centered on an intelligent educational platform, the system must also satisfy intelligence-related requirements. The first is task-sensitive support. The platform should not treat every

prompt the same way. It must identify the likely type of help the student needs and adjust the response pathway accordingly.

The second intelligence requirement is structured tutoring output. The system should return responses that help learning, not just answers. This means the output should include explanation, reasoning, examples, or debugging guidance when appropriate. The third requirement is future personalization support. Even if the current stage of the project does not fully implement learner modeling, the architecture must leave room for features such as knowledge tracing, topic weakness tracking, and student-specific adaptation.

5.1.6 Core Use Cases

The initial use case core is a concept explanation. Here, the student requires assistance in learning something, and the system will provide a clear comprehension, and in case they require, then the examples or the stepwise logic are given. Code tutoring is the second application, based on the student requesting assistance with implementation or getting a better grasp of a programming technique.

The third application is the debugging support. In this case, the student submits a bug, code problem, or faulty logic, and the system gives the explanation of how this can happen and what can be done to rectify. The fourth application is the advanced reasoning support that involves a student posing a more challenging and technical multi-step question, and the system will offer more detailed structure-based assistance. The fifth application is fast clarification, with the system taking a lighter response pathway to short and direct queries when they are related to education.

5.1.7 Data and Storage Requirements

There should be support in the system to store datasets, prompt templates, model adapters, benchmark files, evaluation outputs, and logs of interactions. This is needed as the project has development-time and runtime elements. The data and model artifacts should be structured in a clear way during the development process to enable experimentation to be reproducible. The platform must be capable of logging and recording results during operation, which can be analyzed and optimized in the future.

The design of the storage should also be modular. The model checkpoints, adapters, prompt configuration, and output logs should be considered to be independent assets. This will assist in maintaining the platform organized and more easily update one component of the system without having to update the rest.

5.1.8 Constraints and Assumptions

The system requirements were shaped by several practical constraints. The first is compute limitation. Because the project uses multiple open-weight LLMs, experimentation and inference must remain feasible within academic resource limits. This is why efficient fine-tuning and quantized loading were treated as important design choices.

The second constraint is project scope. The platform is designed for technical tutoring and adaptive support, but it is not yet a fully deployed, large-scale educational product. The third constraint is

evaluation realism. While the project includes strong benchmarking and output validation, the system has not yet been tested at long-term classroom scale with large numbers of real users.

The design also depends on several assumptions. One assumption is that student queries can be categorized meaningfully enough to support routing. Another is that different models provide different strengths that can improve the overall tutoring system. A third assumption is that future learner-aware adaptation can be added to the current architecture without major redesign.

5.2 System Design

The Personalized Education Agent architecture transforms the empirically tested model strategy and system requirements into a practical structure that is pragmatically operationalized. It aims to build a tutoring platform that supports the various modes of educational interaction and maintains modularity, efficiency, and extensibility. The system does not send all requests to a single model because it uses separate tasks to send requests to the most suitable model pathway. As a result of this, the platform fits much better into the actual student needs, where some questions need to be clarified quickly, and some require a more detailed explanation, the use of codes, or an advanced line of thinking.

The architecture takes a layered hierarchy, which sees to it that all the key functions of the platform are clearly separated into compartments. This breakdown leads to maintainability and understanding, empirical verification, and subsequent extension. In addition, it also supports possible future upgrades such as profiling learners, knowledge tracing systems, analytics dashboards, and additional model adapters without requiring the platform to be overhauled.

5.2.1 High-Level Design Approach

The topical system architecture is geared towards the course of a student's query. A user interface receives a prompt submitted by a student, and the application layer receives the request and prepares the request to be processed, and sends it to a routing layer. The routing layer identifies the nature and the complexity of the request and chooses the most appropriate model or a pair of models. The selected model comes up with a tutoring response, which is then sent to the user in a structured format. At the same time, the system records the appropriate metadata to be evaluated, analyzed, and improved further.

The reason behind this architectural decision was that it allows achieving the main goal of the project, which is to offer adaptive educational support. Although the simplified structure of using one model in all the tasks may be easier to apply, it would not be able to capitalize on the unique strengths of the different models. The layered approach, consequently, allows the system to integrate several models in a practical and manageable fashion.

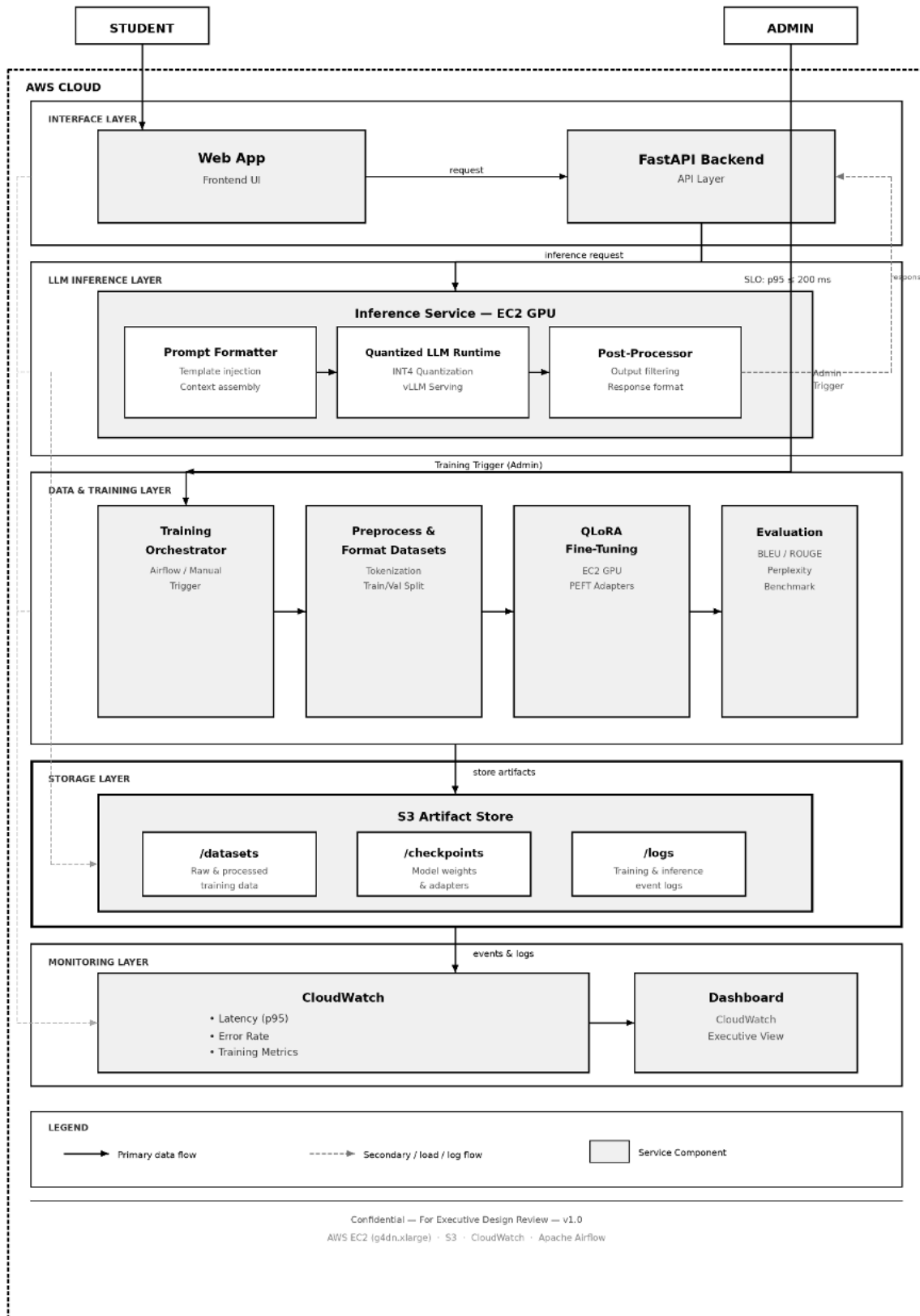
5.2.2 Architectural Layers

This system can be characterized with the help of five main layers. The first one is the user interaction layer, where the students make queries and get responses. The second is the application layer, which receives requests, creates responses, and mediates communications between components. The third layer

has routing and orchestration, which adjudges what model should be used to deal with a given task. The fourth expresses the model inference, where the chosen models and their well-adjusted adapters are stored. The fifth layer is the storage and monitoring, which deals with the controls of data, logs, outputs, and evaluation artifacts.

All the layers serve different purposes in the system. The user interaction layer is user-friendly. The system flow is maintained through the application layer. The routing layer isosomes decision making with tasks. The educative response is produced by the inference layer. Reproducibility, logging, and prospective analytics are supported by the storage layer. It is a functional segregation, which makes the design a strength because it encourages a structured organization and scalability.

AI TUTORING SYSTEM — AS-BUILT ARCHITECTURE



5.2.3 User Interaction Layer

The interface level is the user interface where the learners will interact with the tutoring platform. Such primary input modalities are natural language queries, but the platform can also accept code snippets, debugging prompts, and other technically oriented queries, which require a more structured line of reasoning to answer. The interface should not be complicated and should be readable because the main purpose of the project is to provide tutoring assistance and not to give serious UI design requirements.

The responses provided to the learner are expected to conform to an educational schema. In conceptual inquiries, the system must offer well-developed explanations and examples; in queries connected to the code, it must render codes and dissect the logic behind that code and offer directions on how to implement the code; in queries linked to debugging, it must describe the problem identified and explain how to remedy the issue. In turn, the interaction layer is not just a textual output, but it is also a part of the pedagogical experience.

5.2.4 Application and Control Layer

The query is received by the application layer and validated, given metadata, and forwarded to the routing pipeline. After generating the responses, the output is directed to the interface in a user-friendly format coined by the same layer. The significance of this layer is that it separates the user-facing communication and model-level execution.

In addition, it provides the scaffolding framework that will be used in the future. Request logging, session control, user-level settings, and service APIs are examples of aspects that can be configured in this layer without modifying model logic. This increases maintainability and a more professional and system-based overall design.

5.2.5 Routing and Orchestration Layer

The most conspicuous decision-making part of the system is the routing and orchestration layer. It is empowered to determine what type of assistance is needed by the learner and what the best model should provide. This layer makes the platform adaptive by having the selection of the model dependent on the task typology instead of the model being fixed to each interaction.

As an example, a more condensed conceptual elucidation can be sent to a less computationally expensive or learner model, but a programming question is sent to the Qwen pathway. A pedagogical prompt that requires a lot of elaboration is sent to one of the more robust pedagogical models, whereas a multi-step reasoning problem is sent to DeepSeek. This routing logic is what makes the comparative model-evaluation exercise into a true intelligent system architecture.

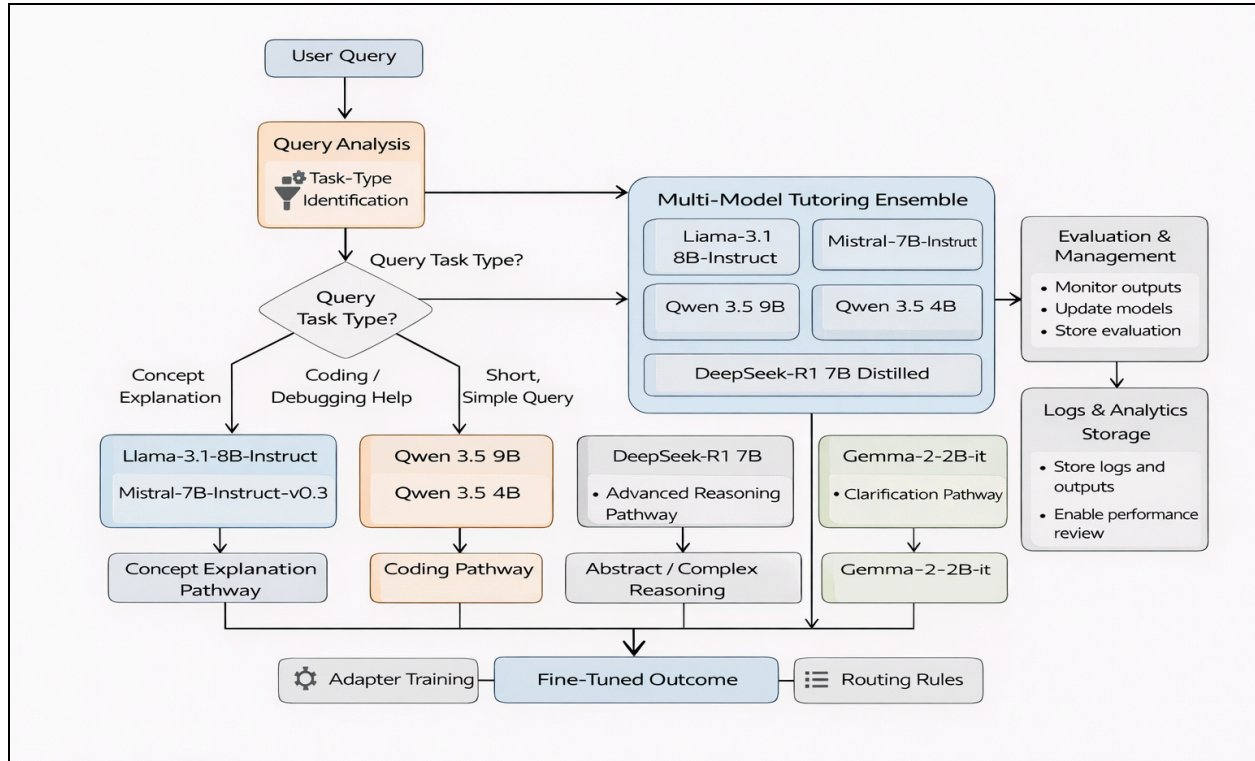


Figure 5.2 Query Routing and model Selection Workflow

5.2.6 Model Inference Layer

The language models used are stored in the inference layer of the platform. It has Mistral-7B-Instruct-v0.3, Llama-3.2-3B-Instruct, Llama-3.1-8B-Instruct, Qwen 3.5 4B, Qwen 3.5 9B, Gemma-2-2B-it, Gemma-2-9B-it, and DeepSeek-R1 7B Distilled. The models are attributed to a particular role depending on the outcomes of empirical validation and comparison with other models.

The inference component also facilitates the adapter-based tuning and structured prompting techniques. When a model is selected, the system builds a tutoring-oriented prompt and makes it available to the model to generate outputs, therefore, making sure that the results are in line with the educational aim of the project and not just a generic text completion. The inference layer, therefore, functions as the response engine as well as a major contributor to the quality of education on the platform.

5.2.7 Storage and Monitoring Layer

The storage and monitoring layer will have the responsibility of handling the artifacts and records that will be necessary to support the platform. These include datasets, prompt templates, adapters, benchmark outputs, generated responses, and model usage logs. It is also important to put these assets in an orderly form to ensure reproducibility, ease in evaluation, and facilitate improvements in the future.

Mechanisms of monitoring system behavior are also offered in this layer. As an example, it could keep a record of which model served a specific request, the time at which the response took place, and what type

of query was provided. The data obtained can then be used in analytics or debugging, or it can be used to refine routing strategies. Even though the present project focuses on report design and experimentation, this layer significantly enhances the general architecture, making it stronger and more advanced.

Table 5.3: System Components and Their Functions

System Component	Main Function
User Interaction Layer	Accepts student queries and displays tutoring responses in a readable format
Application Layer	Manages request flow, input handling, and response delivery
Query Analysis Module	Identifies the type and complexity of the student query
Routing Layer	Selects the most suitable model pathway based on task type
Mistral Pathway	Provides balanced instructional support for general tutoring tasks
Llama Pathway	Supports pedagogical explanation and concept-focused tutoring
Qwen Pathway	Handles coding help, debugging, and technical implementation support
Gemma Pathway	Supports lightweight and efficient educational interaction
DeepSeek Pathway	Handles advanced reasoning and complex technical prompts

Prompting Module	Formats input into tutoring-oriented prompts before inference
Inference Layer	Runs the selected model and generates the educational output
Response Formatting Module	Organizes the generated output into a learner-friendly structure
Storage Layer	Maintains datasets, adapters, outputs, and system artifacts
Logging Module	Records query history, selected model, and response metadata
Monitoring and Evaluation Layer	Supports performance review, validation, and future system improvement

5.2.8 End-to-End System Flow

The flow of the system starts when a student sends a query and ends when the student receives the reply. The request is sent to the routing component by the application layer. The routing logic categorizes the prompt depending on its type and level of difficulty, and it selects the most appropriate model. The tutoring-based response is created with the help of the inference layer in the selected model. The application layer then encodes the output and gives it back to the student. Meanwhile, metadata and records of response are stored at the layer of monitoring.

This flow is significant since it demonstrates that the platform is not just a model demo. It is an organized tutoring system in which the role of every aspect of the process is specified. Future expansion is also supported by the design because once personalization is added later, educator dashboards can be integrated without disrupting the current workflow.

5.2.9 Design Justification

The reason as to why the system design is justified is simply because the system design is equivalent to the real findings of the model assessment. No one model was perfect for all the educational tasks. Some are faster, some that reason, some code, and some balanced instructional output. A multi-model architecture is thus more efficient than a single-model architecture.

It is also system justified in terms of design. It is extensible, clear, and modular. Independently-improvable layers can be added, and the platform can be extended over time with the addition of more models, more robust routing logic, or features that are aware of the learners. This not only suits it to the present project, but it also forms a good basis for further development.

5.3 Intelligent Solution

The intelligent solution of the Personalized Education Agent is based on the use of the adaptive model, structured tutor output, and future-ready personalization. The reason for this solution is to make sure that the system does not have the same response to all academic queries. Instead, it needs to understand the type of help that a student requires and generate a response that is both technically accurate and educationally helpful. This is what makes the platform an intelligent tutoring system as opposed to a question-and-answer interface.

The solution is developed from the notion that various educational tasks demand different behaviors of models. A short explanation does not require the same amount of processing as a complex reasoning problem does. A debugging prompt should not be dealt with in the same manner as the explanation of theory. For this reason, in addition to the intelligence that comes from the selected models, the system is also intelligent in terms of the way that the system reads the tasks, the routes to follow, and the final responses.

5.3.1 Core Intelligence Objective

The main objective of the intelligent solution is to give the right kind of support for the type of student query. In practice, this translates to the system having to detect if the user needs concept explanation, coding help, debugging help, or if he needs deeper reasoning help. Once the task is identified, the platform should be able to generate a response to match that educational need.

This objective is important because the learning process for a student is not one-dimensional. Some learners need short and direct help but others require step-by-step explanation. The system must therefore be flexible enough to allow for more than one learning situation; otherwise, it would lose the consistency in the quality of responses.

5.3.2 Multi-Model Tutoring Strategy

The project's principal intelligent strategy is proposed to be a multi-model tutoring design. Instead of using a single model to respond to each query, the system has different models perform different roles. This helps improve the quality of education as well as efficiency in the system.

In gemma-2-2B-it are introducing lighter models Llama-3.2-3B, and in gemma-2-2B, which support lower and less latency-consuming interactions. richer explanation and better reasoning depth are possible with the stronger models such as Llama-3.1-8B-Instruct and Gemma-2-9B-it. Qwen 3.5 4B and Qwen 3.5 9B are used for coding and debugging purposes, whereas the larger version is dedicated to more

challenging technical prompts. DeepSeek-R1 7B Distilled Explicit Reasoning Tasks
Mistral-7B-Instruct-v0.3 Balanced Instructional Model can support general tutoring scenarios.

This strategy is one of the best sections of the system as it is based on the actual differences found in the model behaviour. And instead of making one model do everything and any job gets assigned to it, the system uses each one where it's the one that's able to add the most value.

5.3.3 Query Understanding

The first step of the intelligent solution is query understanding. When a student provides a prompt, the system must be able to decipher what type of task it is. This includes determining whether the time prompt is concept-focused, code-focused, debugging-oriented, or reasoning-heavy. In future extensions, this layer can also take into account the topic area, level of difficulty, and learner context.

This stage is important because the quality of the final response is closely tied in with correct understanding of the task. A system that blindly routes queries would be wasting the strengths of the model set. When the likely educational purpose of the prompt can first be identified, the platform becomes more adaptive and more useful.

5.3.4 Adaptive Routing Logic

Once the query is understood, the next step is routing. The routing logic determines which model should take care of the query. This is the operational center of the smart solution since it converts the results of the model comparisons to actual tutoring behavior.

Simple conceptual queries can be directed to more modest models so that they are efficient. Coding and debugging queries may be diverted to the Qwen pathway. Explanation-heavy educational prompts can be taken care of by stronger pedagogical models like Llama-3.1-8B-Instruct or Gemma-2-9B-it. More tough technical reasonings can be sent to Deepseek R1 7B distilled. Mistral-7B-Instruct-v0.3 can be used in balanced general instructional responses where neither extreme efficiency nor deep specialization is needed.

This routing logic is important because it enables the system to match the cost and style of response to a computation to the actual educational need. It also provides a good route for future developments, such as confidence-based routing or learner-aware model selection.

Table 5.4: Model-to-Task Routing Strategy

Query Type	Assigned Model(s)	Routing Purpose
------------	-------------------	-----------------

Short concept clarification	Gemma-2-2B-it, Llama-3.2-3B-Instruct	Fast and efficient support for simple educational queries
General concept explanation	Mistral-7B-Instruct-v0.3, Llama-3.1-8B-Instruct	Clear instructional responses for concept-focused tutoring
Deep conceptual reasoning	Llama-3.1-8B-Instruct, DeepSeek-R1 7B Distilled	Stronger reasoning and richer explanation for complex prompts
Coding help	Qwen 3.5 4B, Qwen 3.5 9B	Code generation, logic explanation, and programming support
Debugging assistance	Qwen 3.5 9B, Mistral-7B-Instruct-v0.3	Technical error analysis and correction guidance
Advanced technical questions	DeepSeek-R1 7B Distilled	Structured reasoning for difficult multi-step problems
Balanced general tutoring	Mistral-7B-Instruct-v0.3	Moderate-depth tutoring where neither extreme efficiency nor deep specialization is required
Richer educational response	Gemma-2-9B-it, Llama-3.1-8B-Instruct	More detailed support when the prompt requires stronger instructional depth

5.3.5 Structured Response Generation

After model selection, the system generates the final tutoring response using a structured prompt design. This means the platform does not simply send raw student input to the model. Instead, it frames the request in a way that encourages educationally useful output.

Depending on the task, the response may include concept explanation, reasoning steps, worked examples, code generation, debugging guidance, or a concise summary. This structured generation process is important because the goal of the system is not only to answer correctly, but to support learning. A technically correct answer is not enough if it is confusing or poorly organized.

Structured prompting also improves consistency across different model pathways. Even though the system uses several models, the outputs can still follow a shared tutoring style, which makes the platform more coherent for the student.

5.3.6 Educational Output Design

In response to the selected model, a structured prompt design is used to generate the final tutoring response from the system. This means that the platform is not sending raw student input to the model. Instead, it makes the request in such a way that it promotes educationally useful output.

Depending on the task, the response can contain explanations of concepts, steps of reasoning, generating code, debugging instructions, or simple summaries. This structured generation process is important because the goal of such a system is not just to get the answer correctly, but to aid in learning. A technically correct answer is not sufficient if it is confusing and not arranged properly.

Structured prompting is also another way to support consistency between different model paths. Even though several models are used in the system, the outputs can still be based on a common tutoring style, which makes the platform more coherent for the student.

5.3.7 Support for Future Personalization

Although the current project focuses primarily on the multi-model tutoring core, the intelligent solution was designed in such a way that the goal of offering a system that is personalized can easily happen down the road. In later versions, the system can include knowledge tracing, student history, repeated error patterns, and tracking of topic weaknesses. These signals can affect routing as well as response generation.

For example, a learner who struggles with a concept repeatedly could be given more scaffolded explanations, while a learner who just needs to be reviewed quickly could be given shorter answers. This makes the current architecture the perfect basis for further adaptive learning support to be built, even if full personalisation is not implemented at the current stage.

5.3.8 Why This Solution Is Intelligent

The solution is smart in the sense that it adds a degree of decision-making and role specialization to the tutoring pipeline. A simple system based on the LLM would feed all the prompts to a single model and just return whatever it generates. This project takes a further step of determining the task, mapping the model that offers the best fit to the task, designing the prompt leading to education output, and preparing the system for potential learner-aware adaptation in the future.

That combination of understanding the query, routing the query, generating the query in the structure, and using that model in a modular way is what gives the Personalized Education Agent its intelligence. The value of the platform is not derived from any one model, but rather it is a combination. It finds its origin in the way the system coordinates them in order to provide better educational support.

5.4 System Support Environment

The system support environment specifies the technical foundation needed to develop, evaluate, and operate the Personalized Education Agent. While the previous sections explained the models, architecture, and intelligent routing logic, this section explains the environment in which these components become practical. A strong tutoring system is not only dependent on model selection, but also on strong computational support, software tools, storage organization, and evaluation infrastructure.

The support environment used for this project has been developed to meet two principal objectives. First, it had to sustain effective experimentation over several open-weight LLMs of varying sizes and roles. Second, it needed to be structured enough to be open to extension into a more complete tutoring platform. For that reason, the environment itself was designed as a modular approach, which supports preprocessing, fine-tuning, benchmarking, inference, logging, and future integration into the system.

5.4.1 Compute Environment

The reason for using GPU-enabled execution was that the selected model set includes 2B, 3B, 4B, 7B, 8B, and 9B-scale instruction-tuned models. Running these models for fine-tuning, evaluations, and inference would not be possible in a simple CPU-only setup. The environment was therefore built with a focus on fast execution and support of modern Deep learning workflows.

To make the experiments feasible within the academic resource limit, the system was also built on efficient model loading strategies. This included supporting the execution of lower precision and Christianity runtime management for larger models. These choices were important because the project is a comparison between multiple models instead of focusing on just one. The compute environment, therefore, had to be capable of supporting the running of a series of experiments, rather than testing one time.

5.4.2 Software and Development Stack

The software stack was developed around a machine learning environment using Python. This supported the tasks of model loading, tokenization, dataset handling, fine-tuning, inference testing, evaluation and collection of results. The development process was also based on experimentation on notebooks, which proved easier to test different configurations of the model, comparing results obtained across the chosen set of models.

The stack was used because it allows both research-style experimentation and system-level design. On one side, it has enabled training and evaluation workflows to be run efficiently. On the other hand, it is in

favor of subsequent movement towards a more application-oriented tutoring platform. This benefit of balancing experimentation with the system's readiness is one of the support environment's strengths.

5.4.3 Fine-Tuning Support

A significant component of the environment is parameter-efficient fine-tuning. Since the project contains a number of large models, retraining them completely will be too expensive and not necessary. Instead, the system was designed to use adapter-based tuning in order to allow the selected models to be aligned with educational tasks while retaining fixation of core weights.

This approach helps to save memory cost, time of experimentation, and facilitates maintenance. It also provides the project to better manage tuned variants that are specific to the model. In the context of the Personalized Education Agent, this is particularly interesting since some models might be faster if different tuning priorities are required for their use or not, depending on whether it is being used for concept explanation, coding support, or reasoning-heavy tasks.

5.4.4 Quantization and Inference Support

The environment also includes assistance with the quantized model execution. This was necessary because a number of models chosen are of a size that it is challenging to carry out full-precision experimentation in a limited-resources setting. Quantization helped reduce memory requirements and made it easier to evaluate and compare the increased models without compromising the feasibility of the whole project.

This support also helps to make the final system design more realistic. Since the tutoring platform is designed to service different types of queries with different models, the efficiency of the inference is important. A support environment that only works under good hardware locations would debilitate the project. By adding quantized inference support, the system becomes more practical and in line with deployment-oriented thinking.



Fig : Inference speed across different models

5.4.5 Data and Artifact Support

The system support environment is also composed of an organized storage for datasets, prompt templates, adapters, benchmark files, generated outputs, and evaluation logs. This is important because this project involves the combination of model development, model comparison, and system design. Without having a clear artifact management, it would be difficult to maintain the reproduction of the workflow as well as maintainability.

The environment, therefore, treats data, model artifacts, and evaluation outputs as independent managed resources. This makes it easier to update one of the parts of the system without affecting others. For example, one model can be added with a new adapter without having to change the entire evaluation setup. This modularity is particularly useful in a project that wants to compare a multitude of models and create a structured tutoring platform.

Table 5.5: Support Environment Summary

Support Environment Component	Description
-------------------------------	-------------

Compute Environment	GPU-enabled execution for model loading, fine-tuning, benchmarking, and inference
Development Workspace	Notebook-based Python environment for experimentation, comparison, and result analysis
Model Framework	Transformer-based open-weight LLM support for loading and inference
Fine-Tuning Support	Parameter-efficient tuning using adapter-based methods such as LoRA and QLoRA
Quantization Support	Reduced-precision loading for efficient execution of larger models
Data Processing Layer	Cleaning, formatting, and transforming educational and coding data into tutoring-ready inputs
Prompt Engineering Layer	Structured prompt templates for concept explanation, coding support, debugging, and reasoning tasks
Evaluation Layer	Benchmark testing, runtime measurement, training analysis, and qualitative output review
Storage Layer	Organized handling of datasets, adapters, checkpoints, outputs, and evaluation logs
Monitoring Support	Recording model usage, response behavior, and performance-related information

Integration Readiness	Support for connecting models and outputs to routing logic, application flow, and future deployment
System Maintainability	Modular structure that allows updates to models, prompts, adapters, and evaluation workflows

5.4.6 Evaluation Support

A good support environment should also have the tools for model evaluation. In the case of this project, that includes benchmarking, tracking training losses, inference time, verifying results, and comparative analysis. Since the report is a comparison between model families with different strengths, the evaluation environment must be able to support both numerical and qualitative review.

This evaluation supports the link between experimentation in the model and concluding decisions to be made in the system. It makes it possible to judge whether a model is good in reasoning, good in coding support, efficient enough for a light deployment, or good for advanced tutoring tasks. As a result, the support environment is directly related to the ability of the project to justify its multi-model design.

5.4.7 Integration and Deployment Readiness

Although the present project is aimed at the development of the models and the architectural design, the support environment was designed with reference to the eventual deployment. The use of selected models, adapters, prompts, and evaluation outputs is organized to allow connecting to those in the future to a routing layer, application interface, and logging system. This helps the quality of the project as a whole since it demonstrates that the work is not restricted to isolated experiments.

Integration readiness is of particular importance for a project such as this because the end goal is not simply to compare models, but to produce an intelligent tutoring system. The investigation of a well-designed support environment for investing in the setup of an experimental work has the purpose that such work may later be fully transferred to a real implementation platform without full redesign.

5.4.8 Reliability and Maintainability

The environment of support was also aimed at enhancing the reliability and maintainability. Reliability means that the workflows for preprocessing, tuning, evaluation, and inference can be repeated in a consistent manner. Maintainability implies that models, adapters, prompts, and outputs are replaceable without hindering the ease of managing such a system.

These qualities are important because the multi-model tutoring system can easily become difficult to control if the environment is not well-organized. By making the support structure modular and clearly defined, the project not only becomes stronger in terms of technical credibility but also becomes stronger as a long-term service.

5.4.9 Section Summary

In summary, the system support environment for the Personalized Education Agent consists of GPU-based execution, fine-tuning efficient support, quantized inference, artifact management organization, evaluation infrastructure, and future integration readiness. This environment makes it possible to compare multiple models in a realistic way, yet not compromise the feasibility and maintainability of the project.

With the model development and system design sections now finished, the next chapter shifts into execution analysis as well as overall system outcomes.

6.1 Analysis of Model Execution and Evaluation Results

Having developed, validated, and designed the system, the next thing to do is to consider how the models that have been chosen behave during the execution process and what the results of the evaluation are provided to the Personalized Education Agent. This section is aimed not only at repeating the findings, but at setting them in the context of the educational system. The project is set up as a multi-model tutoring platform; therefore, a model analysis will need to state how the models did in terms of such factors as reasoning, coding support, efficiency, and instructional usefulness, and how the results justify the ultimate architecture.

The execution outcomes had indicated that the selected models do take very different approaches, and these variations matter in designing the system. The larger ones were likely to produce more influential reasoning and elaborations, and were more effective at more complicated prompts. Smaller models were higher in efficiency of speed response and lower in cost of implementation. Models based on coding were more useful in programming and debugging activities, while models based on reasoning were more useful in solving technical problems that were difficult to solve. This trend proves that the selection of the role-based model rather than one model should be used in the end system.

6.1.1 Execution Behavior Across Model Groups

The operational investigation showed that there is an observable performance gap between the smaller and the larger language models when they are being executed. Smaller parameter model versions such as Llama-3.2-3B-Instruct, Qwen 3.5 4B, and Gemma -2 -2B were found to have better execution ease, faster response times, and more repeatable lightweight testing. They worked well, especially in instances where tasks were short, focused, and not required to have a large chain of reasoning. As such, such models proved to have the greatest utility in low-latency tutoring.

On the other hand, bigger models Llama 3.1 8B Instruct, Qwen 3.5 9B, and Gemma 2 9B produced more content and could support more challenging educational prompts with depth. Their expoundings were generally more detailed, and they were more suited to the activity that demanded a more adequate intellectual effort or teaching assistance. Consequently, such models were better remedies for conceptually intense tutoring assignments and high-level scholarly investigations.

The performance of the Mistral-7B-Instruct-v0.3 was steady and even during the performance. Although not dominating all the categories, it was able to consistently provide results in several types of tasks, which makes it a strong mediator reference model.

DeepSeek -R1 7B Distilled had shown its best performance on reasoning -intensive prompts, where the quality of the responses would be particularly beneficial to logically challenging educational problems.

6.1.2 Interpretation of Benchmark Outcomes

The results of the benchmark analysis evaluation support the overall trend witnessed at the point of execution. Variants of each family tended to perform well on reasoning and knowledge-based tasks, which tended to be of larger groups than smaller groups. This was especially true in comparisons between Llama and Qwen and Gemma, with bigger models always performing better with more difficult prompts.

These benchmark results do have good explanatory value in terms of the reliance of the system on a variety of small models. Vigorous tutoring assistance requires more than simple truthfulness but requires a degree of organization in logical reasoning, thorough explanation, and the ability to maintain consistency and consistency in responses over many-step answers. The bigger models were more competent in these areas, thus warranting their high position in the architecture of the intricate education interactions.

On the other hand, the benchmark outcomes indicate that less significant models are still relevant. Although not as effective as the large model, they are still useful in the less taxing work and in keeping the workflow efficient. Such dualistic utility is one of the main reasons why a multi-model design should be used in the project.

6.1.3 Fine-Tuning Analysis

The outcomes of the fine-tuning indicate that using parameter-efficient adaptation was effective in aligning the models of choice with the tutoring goals of the research. The latter tuning had more organized, educative, and relevant outputs. This correspondence is especially surprising since the project is not aimed at the creation of general-purpose response generators but rather at the creation of models that will serve as tutors in the first place.

The fine-tuning stage in the case of Mistral 7B Instruct v0.3 had stable learning dynamics and a significant increase in the quality of instructional responses. Measurement scales based on the Mistral workflow also reveal that the model attained better fitment of the text-centric and code-centric tutoring activities. In the case of the Gemma workflow, the experimental findings provide a clear distinction between the smaller and the larger models; the latter produced higher quality results, whereas the former retained high efficiency benefits.

This discussion testifies to the fact that fine-tuning not only succeeds in the technical dimension but also contributes greatly to the educational process of the project. The tuning helped to get the tutoring-like behaviour closer to the ideal, making the models more useful to the final system.

6.1.4 Runtime and Efficiency Analysis

Runtime analysis is one of the most important parts of the project because the Personalized Education Agent must balance educational quality with practical usability. The execution results showed a strong trade-off between richer output and faster inference. Larger models generally required more time and computational support, while smaller models produced faster outputs with lower cost.

This trade-off is not a weakness of the project. Instead, it is one of the main findings that supports the final architecture. If the system used only larger models, it would provide stronger explanations but would be slower and less practical for routine interactions. If it used only smaller models, it would be efficient but weaker on difficult educational prompts. The best solution is therefore to route different queries to different models based on task complexity and expected response depth.

The runtime findings also strengthen the role of quantization and efficient execution support in the system. These techniques make the larger models more practical to evaluate and use, while still allowing the architecture to benefit from their stronger educational performance.

Table 6.1: Execution and Runtime Analysis Across Selected Models

Model	Runtime	Practical Strength	Main Limitation
Mistral-7B-Instruct-v0.3	Moderate	Balanced tutoring support	Not the fastest option
Llama-3.2-3B-Instruct	Fast	Efficient concept support	Limited reasoning depth
Llama-3.1-8B-Instruct	Slow-Moderate	Strong explanation quality	Higher compute cost
Qwen 3.5 4B	Fast-Moderate	Efficient coding support	Less depth than 9B

Qwen 3.5 9B	Slow	Best coding and debugging help	Higher runtime cost
Gemma-2-2B-it	Very Fast	Quick clarification support	Weak on complex prompts
Gemma-2-9B-it	Slow-Moderate	Richer tutoring output	Lower efficiency
DeepSeek-R1 Distilled 7B	Moderate-Slow	Strong advanced reasoning	Less suitable for simple queries

6.1.5 Coding and Debugging Analysis

The execution analysis showed that the Qwen models proved to have special use in coding and debugging support. Qwen 3.5 9B was more successful in technically detailed prompts, providing more in-depth performance in tasks related to programming. Although Qwen 3.5 4B was still convenient in situations when efficiency was the major concern, the bigger variant was better suited to complex implementation and debugging situations.

Another notable performance observed in coding-oriented executions was the Mistral-7B-Instruct-v0.3, which includes a tuning and evaluation workflow that directly uses technical generation metrics. However, even Qwen line remained a more viable choice in specialized code tutoring, and it confirms the idea that programming support must be seen as an independent system functionality, which is not to be handled as an extension of general pedagogical principles.

This result is important since the coding assistance is one of the fundamental functions of the Personalized Education Agent. The analysis of execution is evident in terms of that technical tutoring requires model choices that differ from those used for pure concept explanation.

6.1.6 Pedagogical Output Analysis

Educationally, it came out that better benchmark performance does not in and of itself guarantee high-quality tutoring output. Models that were most effective in the tutoring system were those that incorporated accuracy and explanations that were well structured, clear, and learner-centred. In this setting, Llama318B-Instruct was specifically effective in the delivery of abstract elucidation and systematically structured instructional material.

DeepSeek -R1 -7B Distilled also had a good result on higher-level educational prompts that require an elevated level of reasoning. This was especially beneficial in situations where prompts required a detailed technical breakdown and turn-taking logic. Similarly, Gemma 2 9B it and Mistral 7B Instruct v0.3 produced useful instructional reactions, particularly in situations where a moderate explanatory style was more preferable to specialised rationale.

These results support one of the key conclusions of the project: it is not always the tutoring model that has the highest overall ability, but the one that best fits the needs of the learner. Pedagogical utility, therefore, continues to be one of the leading dimensions of assessment of the performance of a system.

6.1.7 Error Pattern Analysis

The outcomes also showed that there were a number of effective constraints across the model set. Smaller models would be more apt to give responses that were too short or too shallow to attempt challenging educational tasks. The larger models, which are usually stronger, sometimes gave a longer-than-necessary response and would be better controlled in terms of output.

During code-related tasks, there were answers that were technically correct but were not necessarily as educational or as clear as possible. This was a sign that the coding support does not only entail high technical skills but also effective structuring of responses. These error patterns helped in the refinement of the role assignment of the models and the final routing logic of the system.

These limitations enhance the design of the project instead of weakening it. They show how the actual model behaviour influenced the final architecture, and was not influenced just by theoretical expectations.

6.1.8 System-Level Interpretation

Together, the implementation and assessment results establish ample evidence for the overall system of the Personalized Education Agent. The results do not determine that only one model should replace all other models, but it suggests that the models are best used in certain educational functions. This justifies the multi-model routing approach and the fact that adaptive and task-based model selection is the most effective design option for the system.

Besides, the findings support the implementation of a modular architecture. Since models differ in their ability to reason, programming skill, latency, and instructional behavior, the system will gain greater value as it will position them in a synchronized platform, rather than restricting them to a single-model pipeline. As a result, the project is beneficial on the model level and the system level.

6.2 Achievements and Constraints

The Personalized Education Agent demonstrated high results on the model level as well as the system level. Simultaneously, the project was limited by practical considerations in terms of computing resources, scope, and extent of assessment. The need to express the achievements and the limitations is crucial since it shows that the report is based on the actual developmental circumstances and not

hypothetical assumptions. The strong project does not exist in the context of declarative perfection but of evident progress that is accompanied by a clear definition of the factors that determine its design.

Together, the project managed to create a multi-modal education support structure, which combines the conceptual explanation, coding help, debugging support, guided reasoning, and future proactive personalization design. In addition to the comparative analysis of models, the project outlined a layered system architecture and routing-based intelligent solution, and the functional support environment. Such accomplishments make the project better than a plain model-benchmarking project and make it a realistic tutoring-system design.

6.2.1 Major Achievements in Model Development

Another of the chief achievements of the work was the strict selection and comparative analysis of many open-weight instruction-tuned models to perform in different instructional functions. Instead of focusing the research on one model or architecture, the report compared the Mistral, Llama, Qwen, Gemma, and DeepSeek families in a formal framework that stressed reasoning ability, coding support, computational performance, and pedagogical applicability. This complex comparison served as a better basis for the final system architecture.

The second important success was the implementation of the parameter-effective fine-tuning methods to fit the selected models to the tutoring-based tasks. The findings proved that the models were quite suitable to be adjusted to the production of responses to education without the necessity of thorough training. Output structure, task alignment, and instructional value were improved through the fine-tuning process, especially in workflows that involved direct model-adjustment and systematic assessment. This is a testament to the possibility of educational specialization when the conditions are such as to pose a difficult academic research setup.

The third success was that a full-scale validation framework was developed. This project also avoided a single benchmark or metric, but it combined benchmark testing, runtime performance testing, fine-tuning diagnostics, and pedagogical review. This combined evaluation enhanced the validity of results and gave a valid explanation of the role-based implementation of models in the ultimate architecture.

6.2.2 System Design Achievements

A high achievement at the system level was the construction of a stratified architecture of the Personalized Education Agent. The project has clearly outlined the key components of the system, which include user interaction, application control, routing, model inference, and storage support, providing the report with a systems-wide view and showing that the effort is not limited to stray model experiments.

The other remarkably successful accomplishment was the development of task-conscious routing as the basis intelligent design principle. Instead of forcing one model to answer all queries, the system identifies the type of query that is made by the student and forwards the query to the best suitable model pipeline. Such a strategy improves not only its performance but also its educational value, as well as empirical behavior as seen in evaluation, where different models had different competencies based on concept explanation, code, debugging, and reasoning problems.

The system was also made to be modular to the core. Models, adapters, prompts, logs, and evaluation outputs were considered as discrete parts of the architecture, and this has helped enhance maintainability as well as simplify future refinement. Other potential extensions, e.g., learner-aware personalization, educator dashboards, and increased coverage of subjects, can occur via this modular design, as well.

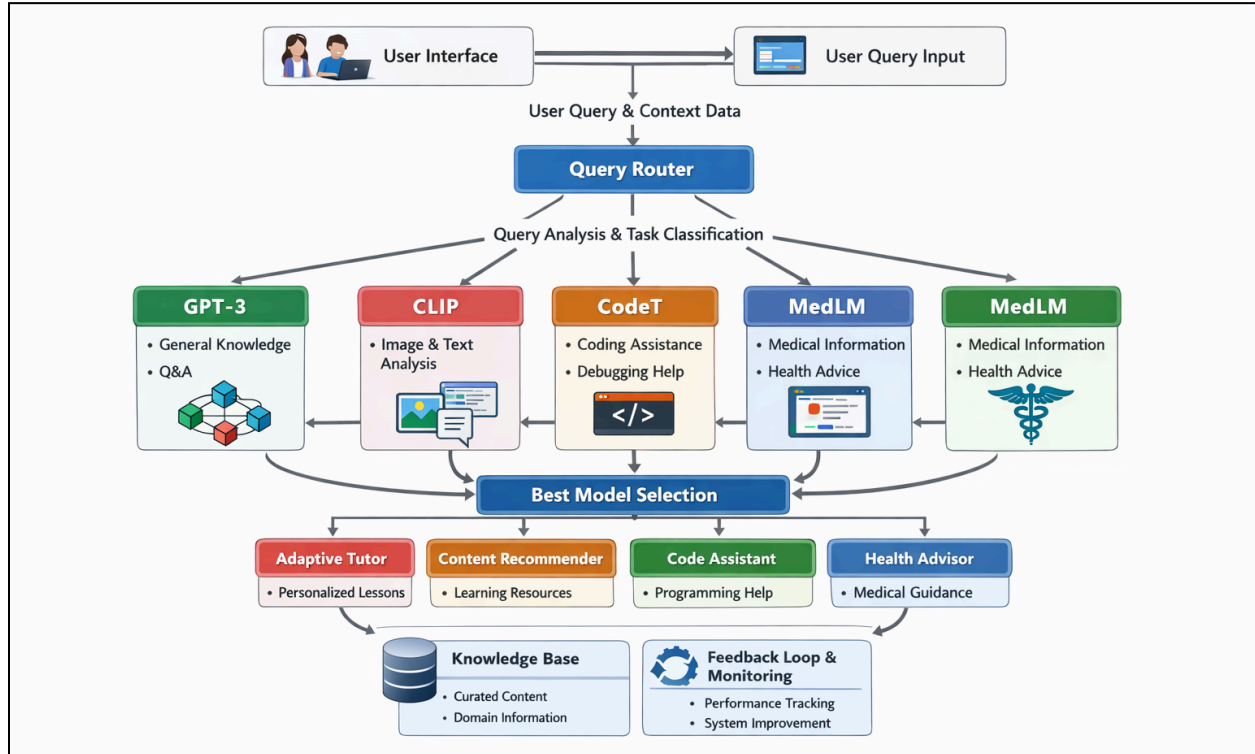


Fig 6.2 Full System Architecture of the personalized Education Agent

6.2.3 Educational Achievements

Educationally, the project managed to make the transition of generic large language model (LLM) response generation towards one based on specific tutoring-oriented support. The models chosen were tested and also classified based on their ability to produce pedagogically useful results as opposed to a healthy production of the correct text. This is a great achievement because the usefulness of the project is based on the aspect of offering meaningful learning support rather than referring only to technical generation fidelity.

In addition, the project was successful in covering a variety of tutoring situations under one system architecture. It provides features like concept presentation, code advice, debugging support, and work involving a lot of thought. The scope of such abilities is remarkable as it is in line with the genuine student needs in technical education. Furthermore, the systematic distribution of the different roles to the different models improves the design of education, whereby the functional support can be matched to the model functions more accurately.

6.2.4 Practical Achievements

One of the most crucial project outcomes was that it conformed to a realistic feasibility. The selected workflows used effective methods of hyperparameter tuning, support of model quantization, and regular assessment protocols, avoiding the use of unfeasible full-scale retraining. As a result, the validity and the usability of the study in real project situations are significantly increased.

In addition, a workflow that is reproducible and extensible was obtained in the project. Preprocessing of data, hyperparameter optimization, system evaluation, and interpretation of data were not viewed as a discrete experiment. This integration supports an easier exposition of the system, better testing of the system, and simpler future development.

Table 6.2: Major Achievements of the Project

Achievement Area	Major Achievement
Model Development	Selected and compared multiple open-weight LLMs for tutoring, coding, and reasoning tasks
Fine-Tuning	Applied parameter-efficient tuning to improve educational response quality
Evaluation	Built a multi-layer evaluation framework covering benchmarks, runtime, and pedagogical review
Coding Support	Established a dedicated coding and debugging pathway through the Qwen models
Reasoning Support	Added a specialized reasoning pathway using DeepSeek-R1 7B Distilled
System Design	Designed a layered multi-model tutoring architecture
Intelligent Solution	Introduced task-based model routing for adaptive educational support

Efficiency	Balanced stronger models with lightweight models for practical deployment
Data Workflow	Structured data preparation and artifact management for reproducible experimentation
Future Readiness	Designed the system to support future personalization, analytics, and expansion

6.2.5 Core Constraints

Although this project reported strong performance, it had a number of feasible limitations. The major limitation was on computational availability. Since the chosen models consist of several 7B, 8B, and 9B, it was necessary to conduct experiments within the realms of academic resources. This constraint also affected the scope of tuning and evaluation processes that were possible, which highlights the need to have effective methodological practices.

One of the secondary limitations had to do with the extent of the project. Though the initial concept was of an all-inclusive tutoring system, the current stage of development is focused on the basic technical learning operations and the general underpinning of personalization, and not on fully-fledged production implementation. Thus, the design is already rather strong, but it has not yet been tested as a part of a large-scale and real-user learning setting.

The third weakness is associated with the depth of evaluation. The project uses both benchmark-based analysis and qualitative review; however, real educational systems are, per se, difficult to analyze in their totality by means of short-term experimentation only. Systematic learner outcome tracking, longitudinal classroom validation, and extensive user testing go beyond the current scope.

6.2.6 Model-Specific Constraints

There were also constraints created by each model cohort. Smaller systems like Llama-3.2-3B-Instruct and Gemma-2-2B-it were effective but could frequently not be deep enough to handle tutoring tasks that require reasoning-intensive tasks. On the other hand, bigger models, such as Llama-3.1-8B-Instruct, Qwen-3.5-9B, and Gemma-2-9B-it, produced more powerful results, though they required more computational power and were less efficient in lightweight interactions.

Within the area of coding-oriented assessment, some outputs were technically correct, but failed to follow pedagogical structuring as necessary. This observation shows that strong coding skills alone cannot be used; the inclusion of educational formatting and explanatory context is an essential part. Although DeepSeek -R1 7B Distilled has shown high scores on higher reasoning prompts, it is not the most

effective option in all teaching situations. These restrictions explain, to some extent, the reason why role-based routing is used in the end system as opposed to the use of only one model.

6.2.7 System-Level Constraints

One of the major limitations at the system level is that overall learner-aware personalization is yet to be established. The architecture being evaluated is meant to facilitate upcoming knowledge tracing and adaptive behavior, but the current project largely achieves a base structure on such goals as opposed to providing a complete accomplishment of a personalized learning loop, with long-term user history being able to be incorporated into it.

The next issue at the system level is associated with the routing strategy, which, though being a sound conceptual and architectural element, still has room to improve. Future work would fine-tune the classification of tasks, use routing schemes that are aware of confidence, and combine learner-state information, which would make the system stronger. The design that currently exists will, therefore, be a significant base, but not the final version of the platform.

6.2.8 Why the Constraints Matter

These are not weaknesses that are hidden. They have, in many cases, directly increased the design. The efficiency of tuning and quantization has been encouraged by computational limits. The difference in the capabilities of models has led to a multi-model architecture. The lack of convergence between pedagogical and coding output has helped in establishing better-defined model roles. This absence of absolute personalization at this stage has inspired the design of an extensible architecture as opposed to a lock-in architecture.

Therefore, these limitations have helped to develop a more lifelike and improved structured tutoring platform. They have made the end design more considered and believable.

6.3 System Quality Evaluation of Model Functions and Performance

The general quality of the Personalized Education Agent was tested by how well the system appropriately performs the education they were designed for, but also how well they operate in practice. This section is focused on the quality of the system as a complete tutoring platform and not only as a collection of individual models. The goal is to determine whether the architecture, routing strategy, and model selection provide a useful but also reliable, efficient, and susceptible to future extension.

System quality in this project was assessed based on two major dimensions. The first is functional quality, which is a measurement of whether the system can successfully assist the tasks for which it was designed. The second is performance quality, which is how well the system performs those tasks, in terms of response usefulness, runtime behavior, consistency, and maintainability. These two dimensions are important since an educational platform needs to be capable as well as practical.

6.3.1 Functional Quality of the System

The first aspect of the quality of systems is functional correctness. The Personalized Education Agent was designed to help support concept explanation, code tutoring, debugging assistance, and advanced reasoning support. Judging by the validation of this model and the system design, the solution has successfully met those core functional goals. It can take in various kinds of educational prompts, understand what kind of job it is likely to be and produce answers that are in accordance with the desired tutoring objective.

The system has also shown its functional flexibility. It does not retrieve all queries equally. Here, it supports more than one type of task via a routing-based design. This is a good quality strength in terms of enabling the platform to act more like an adaptive educational assistant, and less like a 'response-generating' bot. Functional quality is thus supported not only by the model capability, but also by the organization as to how the system organizes and uses that capability.

6.3.2 Quality of Educational Responses

A huge measure of the quality of the system is the utility of the responses generated. In this project, a high-quality educational response is technically correct, clearly structured, and relevant to the prompt, which will help learning. The strongest responses were those that included correctness as well as explanation, reasoning flow, and a format that was easy for students to follow.

The reason why the system worked well in this respect was that the response generation system was designed in such a way that tutoring was central, not just answer completion. Concept explanations were strongest when they turned the topic into easy-to-understand lessons. Coding responses were greatest when responses explained the logic of implementation rather than a mere code response. Debugging responses were strongest when they identified the problem, explained the cause, and suggested the fix understandably. These qualities have demonstrated that the project found the right balance between the system output and educational value.

6.3.3 Model-to-Task Fit

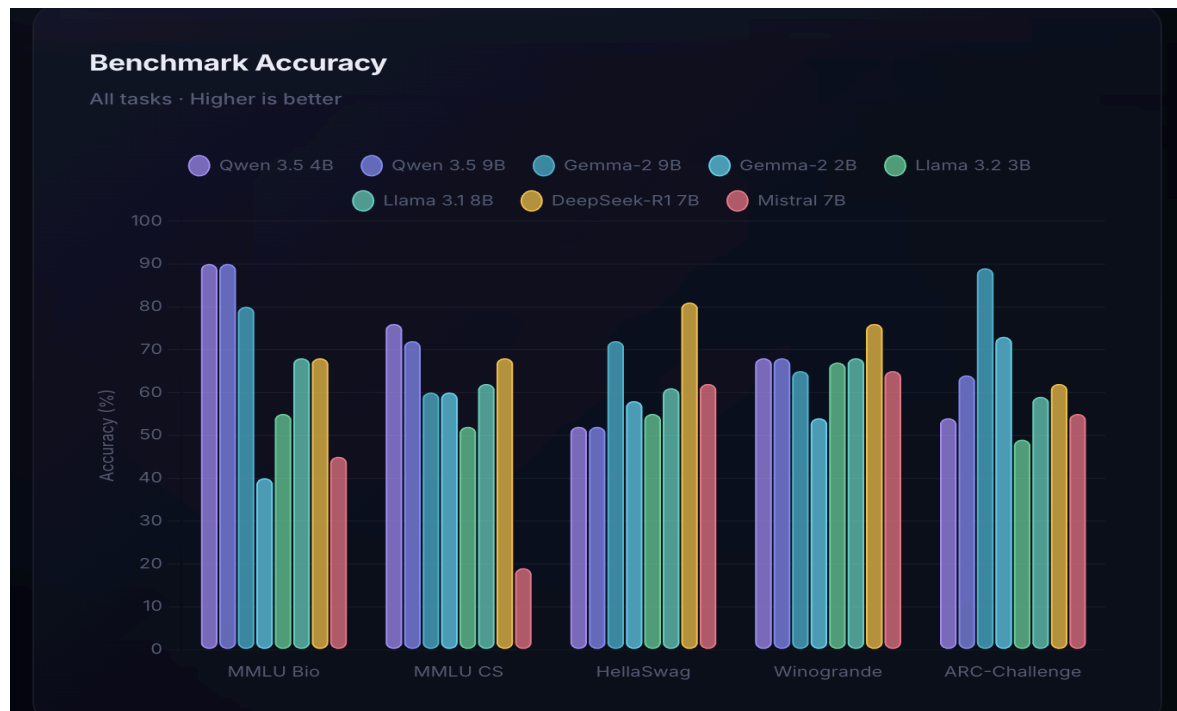
Another aspect crucial to system quality is the degree of fit between each model and its role assigned to it. The Personalized Education Agent is predicated on the notion that there are different models of tutoring tasks that are better suited to different types of tutoring tasks. The quality of the platform, therefore, does not only depend on the single strength of models, but also on whether the routing strategy assigns them appropriately.

This role-to-task fit is one of the strongest parts of the system. Mistral-7B-Instruct-v0.3 works well as a balanced model of instruction. Llama-3.2-3B-Instruct and Gemma-2-2B-it is suitable for lower-cost and efficient support. Llama-3.1-8B-Instruct and Gemma-2-9B-it provide better reasoning and more prolific explanation. Qwen 3.5 4B leverages the strongest coding and debugging way. DeepSeek-R1 7B Distilled Supports Difficult Reasoning Heavy Prompts. This alignment helps to make the system more effective in providing education and more efficient in its operations.

6.3.4 Performance Quality

Performance quality is how well the system works, how it is working in practice. A tutoring system proves no good if it can provide a good answer but takes too long, is not efficient with its resources, or is difficult to maintain. The evaluation results revealed a good trade-off between richer responses of larger models and faster responses of smaller models. This trade-off is a central part of the final architecture.

The system works well in terms of performance since it does not squash all tasks through the same model. Lightweight interaction could be managed by smaller models, whereas more complex educational tasks can be forwarded to stronger models. This makes the entire leading usability of the platform and makes the design more practical. Instead of optimizing for one factor, such as benchmark score, or another, such as speed, according to educational need, both are balanced using this system.



6.3.5 Reliability and Consistency

Reliability is another metric that is important in understanding system quality. In this project the concept of reliability refers to the ability of the system to generate reasonably stable and structured behavior under different tutoring scenarios. The architecture enhances reliability as it separates the major stages of the workflow - query intake, routing, inference, and logging. This makes the system easier to manage and contributes to keeping the system consistent from repeated interactions.

The fine-tuning process and the prompt design oriented to tutoring also helped to improve consistency. The models started to be more likely to come up with responses that were useful in an instructional format instead of a purely open-ended format. While generative systems will always exhibit some level of variation, with the project, we achieved a good level of consistency in terms of model role assignment, output structure, and educational alignment.

6.3.6 Maintainability and Scalability

In addition, a high-quality system must be maintainable and scalable. The Personalized Education Agent does well in this regard since it was built in a modular architecture. Models, adapters, prompts, log, and evaluation outputs are considered to be separate parts and not tightly bundled into one difficult pipeline. This makes the updates easier and makes for better long-term usability.

The system is also of a scalable design. New models can be added, new subject areas can be introduced, and future learner-aware components can be incorporated without a complete rebuild. This is important because what educational systems need to do is to evolve. The ability to expand the platform without compromising its structure is a big indicator of quality.

6.3.7 Trade-Off Analysis

The system quality evaluation also indicated some realistic trade-offs. The first is the tradeoff for the depth of reasoning versus the efficiency of run-time. The larger models tend to have a better quality of explanations, but are slower and have a higher resource consumption. The second form of trade-off is between technical specialisation and over-general instruction behaviour. Code-focused models are better at programming, while pedagogical models are better at broader explanation as related to education. The third tradeoff is between simplicity and flexibility. A single model design would be easier, but would not allow the same level of task adaptation.

These trade-offs do not compromise the system. Instead, they indicate that the project made realistic design choices. The final architecture has the ability to give a better quality, because these trade-offs are handled instead of being ignored. This is one of the primary reasons that the system is strong as a whole.

6.3.8 Overall Quality Assessment

When considered in its totality, the Personalized Education Agent shows high system quality in terms of the scope of the project. It does what it is supposed to do as its tutoring functions; it is effective in multiple educational tasks, it effectively utilizes model specialization, and it is practical under academic resource limits. Its architecture is modular, its evaluation process is structured, and its design obviously supports future development.

The best quality aspects of the system include its adaptive routing design, its balance between efficiency and capability, its support for coding, explaining concepts, and its readiness for the future, in terms of personalization. The areas identified for further improvement are more in-depth real-user testing, more sophisticated routing logic, and better integration of learner history. Even considering these future opportunities, the existing system is already a strong and credible intelligent tutoring system.

6.4 System Visualization

System visualization is an important part of presenting the Personalized Education Agent in a clear and professional manner. While in the previous sections, there were explanations of models, architecture, the

evaluation process, and the quality of the systems in written form, in this section, the focus is on how the visual representation of this project should be done in the final report. Good visual presentation enhances readability, better technical communication, and helps the reader get a better understanding of the complete system in a shorter time.

The purpose of visualization in this project is to help with four main goals. First, it should display the architecture of the tutoring platform in a way that makes it easy to understand how the components of the platform interact with each other. Second, it should give the role of each model in the multi-model system. Third, it should be a concise and comprehensible form of evaluation and performance outcomes. Fourth, it should show that the project is a full-fledged educational AI system and not merely a bunch of models containing model experiments.

6.4.1 Architecture Visualization

The most important visual in the report should be the general system architecture diagram. This figure should represent the end-to-end architecture of the Personalized Education Agent, such as the student interface, application layer, routing layer, model inference layer, and storage or monitoring layer. The purpose of this diagram is to illustrate how a student query passes through the system and how the final tutoring response is made.

A second important architecture figure who should pay particular attention to query routing. Since the project is constructed around the role-based model selection, this figure should illustrate the assignment of different types of educational prompts to different models. For instance, concept explanations, code support, debugging, and reasoning-heavy prompts should be visualized on separate tracks in the system. This makes the intelligent solution easier to comprehend and may support the logic of the report.

6.4.2 Model Comparison Visualization

A major part of this project is the comparison of models by capability, efficiency, and usefulness of tutoring. For this reason, the report should contain some visuals that summarize the role and force of each selected model. A comparison table that is well designed can communicate this very well, including the model name, size, main use case, key strength, and main limitation.

In addition to a table of comparison, a visual chart can be used to compare models when it comes to bigger picture aspects like instructional quality, depth of reasoning, coding support, and efficiency. This figure need not be too complicated. It is to make it easy to understand the trade-offs between models. This is particularly useful, given that the report concludes that there is no single model that is best for every task.

Model	Size	Strength	Main Use Case	Limitation
-------	------	----------	---------------	------------

Mistral-7B-Instruct-v0.3	7B	Balanced quality tutoring	General educational support	Less specialized
Llama-3.2-3B-Instruct	3B	Efficient concept support	Short explanations	Lower reasoning depth
Llama-3.1-8B-Instruct	8B	Strong pedagogical output	Deep explanation concept	Higher runtime cost
Qwen 3.5 4B	4B	Efficient coding helps	Fast programming support	Less detailed than 9B
Qwen 3.5 9B	9B	Best technical support	Advanced coding and debugging	Higher compute cost
Gemma-2-2B-it	2B	Very efficient	Quick clarifications	Weak on complex tasks
Gemma-2-9B-it	9B	Richer tutoring output	Stronger educational support	Lower efficiency
DeepSeek-R1 Distilled 7B	7B	Advanced reasoning	Complex technical prompts	Less suitable for simple queries

6.4.3 Evaluation Result Visualization

The report should also include figures that summarize the evaluation and validation results. One useful figure is a benchmark comparison chart showing how the selected models perform across reasoning and technical evaluation tasks. This helps communicate the difference between smaller and larger models as well as the importance of specialized models such as Qwen and DeepSeek.

Another important figure is a runtime or efficiency chart. This should show the trade-off between response quality and operational cost, such as inference time or throughput. This is one of the most important findings of the project, so the report should show it visually rather than only describing it in text.

If available, a training-loss figure or adaptation-progress chart should also be included. This helps demonstrate that fine-tuning improved alignment with the tutoring objective and gives the report stronger technical depth.

MODEL	PARAMS	MMLU BIO	MMLU CS	HELLASWAG	WINOGRANDE	ARC	PERPLEXITY	TOK/S	OVERALL
Qwen 3.5 4B	4B	98%	76%	52%	68%	54%	3.21	5.2	68.8%
Qwen 3.5 9B	9B	98%	72%	52%	68%	64%	2.95	3.8	69.2%
Gemma-2 9B	9B	88%	68%	72%	65%	89%	2.59	3.8	73.2%
Gemma-2 2B	2B	48%	68%	58%	54%	73%	2.88	6.6	57.8%
Llama 3.2 3B	3B	55%	52%	55%	67%	49%	3.45	5.8	55.6%
Llama 3.1 8B	8B	68%	62%	61%	68%	59%	2.78	3.1	63.6%
DeepSeek-R1 7B BEST	7B	68%	68%	81%	76%	62%	2.72	3.4	71.8%
Mistral 7B	7B	45%	19%	62%	65%	55%	2.58	2.8	49.2%

Fig : Every model every metric

6.4.4 Educational Output Visualization

Because the project is educational in nature, it is useful to show example outputs in the report. One figure can present a sample student query alongside the system’s tutoring response. This is valuable because it helps the reader understand how the platform behaves in practice. It also shows that the output is educationally structured rather than simply generated text.

A second useful visual is a conceptual dashboard or monitoring view. Even if a full dashboard is not yet implemented, a simple mockup can show how model usage, query categories, or interaction logs could be presented. This strengthens the system design sections by showing how the platform may be observed and improved in future versions.

6.4.5 Recommended Visualization Set

To keep the report strong without making it too crowded, the final version should focus on a small but meaningful set of visuals. The most important figures are the full system architecture, the routing workflow, the model comparison summary, the benchmark comparison chart, the quality-versus-efficiency chart, and one example tutoring output. These visuals are enough to support the core story of the report and make the project easier to evaluate.

Additional visuals can be added if space allows, but they should only be included if they support a clear point. Each figure and table should have a proper title and should be placed near the section where it is discussed. This keeps the report organized and professional.

Appendix A: System Testing

System testing was conducted to ensure that the Personalized Education Agent performs the actual function of tutoring properly and consistently. The intention of testing in this project was not confined to verifying whether the models can produce outputs or not. It was also intended to ensure that the system can be used for different types of academic queries, can be directed to the appropriate models, and can provide structured educational responses as well as enable a stable end-to-end tutoring workflow. Since the project is implemented in the form of a multi-model educational platform, testing had to cover both model behaviour and system-level interaction flow.

The testing approach has been to bolster the realistic educational use cases, and not just the isolated test prompts. The system was tested against representative scenarios such as explanation of concepts, code tutoring tasks, debugging support, advanced reasoning, and short clarification tasks. These cases were chosen because they represent the significant functions of the platform and can help demonstrate whether this system behaves like a real educational assistant.

A.1 Testing Objectives

The first goal for testing was functional verification. The system was required to demonstrate that it could take a valid student query and provide a meaningful response in line with the intended role of tutoring. The second objective was routing verification. Since the architecture relies on different tasks being assigned to different models, the system needed to demonstrate that the chosen routing logic works as it should in practice.

The third objective was response quality checking. The generated outputs had to be understandable, relevant, and educationally beneficial instead of being technically correct. The fourth objective was a stability check. The system had to maintain reasonably consistent behavior with repeated testing of similar categories of prompts. Together, these goals reflect the need not to assess the project as a compilation of model comparisons but as a working tutoring project.

A.2 Testing Strategy

The testing strategy was a combination of scenario-based testing and reviewing outputs. Each test started with a sample educational query of a realistic student interaction. The system then processed the query through its routing and inference workflow. The generated response for the selected model was analysed for correctness, structure, and suitability for learning support.

This was a useful approach since it enabled the project to simultaneously test architecture and output quality. Instead of testing whether the model generated something, the testing process tested whether the

system acted correctly from input to output. This made the results more relevant for the final design of the tutoring platform.

A.3 Core Testing Categories

The first category for testing was concept explanation. These tests examined the system's ability to provide an explanation that is clear and structured for student questions that have a theoretical basis. The second category was code tutoring. These tests included the aim to see if the platform was able to support programming help, implementation guidance, and algorithm explanation.

The third type was debugging support. In these tests, the system was given prompts with code errors or faulty logic and was expected to explain the problematic issue and offer a correction. The fourth category was advanced reasoning support. These tests saw if the system was able to respond effectively to hard multi-step technical prompts. The fifth category was fast clarification. These tests saw the system being put through its paces in order to see if it could be used to provide short and useful responses for simple educational queries in an efficient manner.

A.4 Functional Test Cases

In one case of the concept explanation test case, the student had asked for help with a technical topic. The system was able to recognize the prompt as one that required an explanation-focused response and chose a model more oriented toward pedagogical response generation. The resulting output was both clear and logically arranged and fit for tutoring use.

In the code tutoring test case, the student was asking for implementation help. The system was identified as code-oriented and passed to the technical model pathway. The output contained some relevant code support and explanation, indicating that the system is able to handle tasks of programming-related tutoring well.

In a debug test case, the student gave a code-related problem and asked for help in debugging it. The system was able to choose the pathway to use for coding and create a response that identified the likely error, explained the problem, and recommended a corrective action. This confirmed that debugging support is handled as a separate tutoring function.

In the advanced reasoning test case, they had to give a deeper and more structured answer. The system directed this request to a pathway with reasoning capability and returned a more detailed answer in the form of an instruction. In the quick clarification case (test), the system was operating with a lighter model path for a shorter educational prompt to demonstrate that lower-cost response handling is also supported effectively.

A.5 Routing Verification

Routing verification was one of the most important part of System testing as the project is dependent on role-based model assignment. The results of the testing showed that concept explanation prompts were directed toward pedagogical model paths, code and debugging prompts were directed toward the technical

model path, and advanced reasoning prompts were directed toward the stronger reasoning-oriented model path.

This confirms that the architecture itself is doing something more than just exposing a number of models. It is making specific use of them. Correct routing can improve the quality, as well as the efficiency, of the system because it ensures that model capability is matched with educational needs. This is one of the most technical results of the project.

A.6 Response Quality Verification

The testing process also analysed the quality generated responses. A response was considered to be successful if it was technically correct, clearly organized, and beneficial for the students to learn something. For concept explanation, the response called for understandable reasoning. For coding support it had to explain the implementation or logic. For debugging, it had to detect the problem and give the learner advice to correct it.

The outputs were strongest when the task and model role were a good fit to each other. This proves the central design idea of the project. The quality of an education does not just depend on the size of the model or its raw capability. It is dependent on good strategy of model selection and controlled tutoring output. This result strengthens the design of the system and supports the final architecture.

A.7 Stability and Consistency Testing

Stability testing was focused on whether the system could have a consistent tutoring workflow under several different scenarios. The tests revealed that they generally maintained structured output behavior, making sensible routing decisions, and gave learner-focused responses during the tests when repeated over and over again. While generative systems intrinsically support some amount of variation, the total pattern of behaviour was at the current project stage stable enough for an educational support platform.

This level of consistency is important as the behavior of a tutoring system should not be unpredictable on similar requests. The project design, where the structuring of the prompts and routing were based on a role, contributed positively to this stability.

A.8 Testing Limitations

The testing process was also, however, limited to some degree. The use of representative scenarios rather than a large scale deployment in an actual classroom was what the system was tested against. This means that this testing sings well for design level and functionality level validation, but for now does not replace the long term user testing. Another limitation is that there are some aspects from larger models that may need more control of the formatting you wish to see on outputs in future versions.

These limitations make sense for the scope of the current project. They do not lower the value of the testing results, but they do establish clear areas for improvement in the future.

Appendix B: Project Data Source and Management Store

The Personalized Education Agent is based on a systematic data basis, which supports model tuning, model evaluation, and system design. Since the platform is designed to provide explanations of concepts, tutoring of code, debugging help, and technical reasoning services, the project required information that represents multiple types of educational interaction. The following appendix will outline the main classifications of data used during the project, preparations of data, and the scheme of organization within which the development of the project and evaluation will take place.

This appendix aims to prove that the project has a clear and transparent data workflow, not only a selected model. A well-developed model performance of an educational system depends on careful data collection, data cleansing, transformation, and storage. In her turn, data preparation and data management were considered as the key elements of the overall system architecture.

B.1 Data Source Categories

The information that was used in the initiative can be classified into four major groups. The former type is concept-oriented learning information consisting of the question-answer format of information, explanation-based content, and topic-based learning examples that are aimed at enabling the models to produce more accurate academic answers.

The second category is associated with the programming and code-related data, including problem descriptions, code solutions, implementation examples, and technical prompts to assist in teaching coding and describing algorithms. It is important that this category is significant since one of the major purposes of the Personalized Education Agent is to offer coding support.

The third type is debugging data, which comprises such data as instances of erroneous code, commonly used programming mistakes, and tasks involving explanations of fixes. Such records allow the models to train to detect problems, explain failures, and provide substantive debugging instructions instead of just providing fixed outputs.

The fourth type entails the evaluation data that contains benchmark datasets and structured testing prompts that one uses to compare the chosen models. These data sets have been deliberately separated from the tuning-oriented data to improve the reliability and the interpretability of the model validation.

B.2 Role of the Data in the System

The system is based on different components that have distinct data categories. Those types of datasets are concept-oriented and thus build the quality of explanation and structure of the pedagogical architectures. The system is also able to provide instruction in technical areas because of its programming-oriented approach. The data that is found in debugging enhances the responsiveness of the system to inquiries that focus on errors. Evaluation data make it possible to compare models fairly and also allow a more objective evaluation of the performance of the system.

This distinction is important, as the project does not depend on a single learning signal, but combines heterogeneous learning requirements in a single architecture. This diversity, therefore, needed to be reflected in the data strategy. A tutoring platform that supports diverse academic functions should also be trained and assessed with data that measures the different functions.

B.3 Data Preparation and Standardization

The pre-use of the dataset in the project entailed the subjecting of the information to cleansing and standardisation processes, thus making the information conform to a specific format. Raw educational and programming data are generally not homogeneous with regard to form, length, quality, etc. Some of the entries are properly formed and easily exploitable, whilst others show part of a text, malformed formatting, repetition of entries, or low informational quality. These shortcomings can reduce the quality of fine-tuning mechanisms and invalidate model outputs.

To overcome these problems, the project used a systematic version of the reprocessing workflow, which cleaned raw records and converted them to a common tutoring-aligned schema. Every viable record was rewritten as a prompt-response paradigm that could be executed with cross-model and, in this way, the comparability of models was maintained, and the resultant outputs were traceable to a standard input preparation protocol.

The standardisation protocol also increased conformity to the pedagogical goals of the project. Instead of leaving the raw data as it is, the workflow converted it into a more pedagogically useful format, which increased the value of the dataset as an input to instructional response generation.

B.4 Data Transformation for Tutoring Use

After the standardization, the data were restructured into forms that supported instructional prompting. Conceptual datasets were converted into question explanation constructs. Dataset programming was structured in such a way that prompts, which include coding difficulties, logical explanation, or advice on implementation, are allowed. The dataset debugging was designed in a manner that would allow the system to be able to describe the etiology of an issue before suggesting remedial solutions.

This change is essential, and the Personalized Education Agent is not set to generate answers in a random and merely conversational way but to generate pedagogically useful content. As a result, the database design was designed in a way that encourages explication, inferential reasoning, illustrative examples, and hierarchical support.

The same rationale also allows consistency across the model pathways. Although the project compared the work of other models, the data transformation also formed a consistent model of instruction that contributes to the overall coherence of the platform.

B.5 Data Quality Considerations

The issue of data quality was considered as one of the primary requirements in the project. The education systems are more vulnerable to poor training examples in which poor value records can create ambiguous,

mistaken, or false tutoring reactions. Therefore, the preparation process puts much emphasis on the ideals of clarity, completeness, relevance, and technical correctness, thus making the instructional material comply with high academic standards.

This issue is of special relevance to the data sets related to the field of programming and debugging, where bad examples may directly impact the performance of the code support systems. In this regard, intensive filtering and data cleansing processes were applied so that it is more likely that the final models would produce responses that are not only technically substantive but also pedagogically coherent. The strong data quality, in its turn, makes it easier to conduct a more credible evaluation protocol and makes it possible to interpret the empirical findings more clearly.

B.6 Data Management Store

The project implemented a data management system in such a manner that datasets, training files, temporarily transformed evaluation files, and the outputs in the course of experimentation were arranged in an orderly manner. In this model, the existence of a special management store is used to separate the concepts of the static resources and the dynamic resources: The former includes raw datasets, cleaned datasets, prompt templates, benchmark files, and persisted adapters, whereas the latter includes generated responses, evaluation logs, inference outputs, and performance records.

This segregation is beneficial because it helps in maintaining the integrity of the development workflow. What can be reused are static resources; such resources can be reused in each of the experimental repetitions, whereas dynamic resources are subject to repetitive updates as new evaluations or model runs are undertaken. This results in improved maintainability and long-term extendability of the project.

With a multi-model ecosystem, a carefully designed management store is essential in order to reduce uncertainty in the datasets, model artifacts, and result files. This also increases the effect of reproducibility because the provenance and purpose of each stored asset are clearly recorded.

Table B.1: Data Sources, Purpose, and Usage in the System

Data Source Category	Purpose	Usage in the System
Concept Explanation Data	Supports clear academic and topic-based explanations	Used for tutoring-style response generation and concept clarification
Programming Data	Supports coding logic, implementation help, and technical guidance	Used for code tutoring and programming assistance

Debugging Data	Supports error identification and correction-oriented responses	Used for debugging help and fault explanation
Benchmark Datasets	Supports objective comparison across selected models	Used for model evaluation and validation
Prompt Templates	Standardizes model input structure across tasks	Used in routing, prompting, and response generation
Training / Validation Splits	Supports controlled fine-tuning and performance checking	Used during model adaptation and validation
Evaluation Outputs	Records benchmark, runtime, and response-quality results	Used for result analysis and report generation
Logs and Generated Responses	Captures model behavior during testing and system use	Used for monitoring, review, and future improvement

B.7 Storage Structure

The storage architecture was developed to support the entire project workflow. Raw source material is maintained so that it can be referred to and traced. The data under is cleaned and transformed into another repository to enable reuse when fine-tuning and testing are being done. Model adapters and checkpoints are stored in a special artifact repository, and evaluation outputs and logs are stored in a results repository.

It is an architecture that supports development as well as reporting. In the course of experimentation, it simplifies workflow management. In the process of writing reports, it simplifies the process of connecting the findings to the relevant model pathways or categories of data sets. Also, the architecture can be extended in the future through the ability to add the learner history, dashboard information, or subject specific version in the future.

B.8 Importance to the Overall Project

The Personalized Education Agent is enhanced in a number of ways by the data source and management design. First, it aids in fair model comparison since all the models would be fed with well-prepared inputs. Second, it enhances the quality of tutoring through the moulding of data concerning the generation of responses to education. Third, it facilitates reproducibility through maintaining the organization of datasets, outputs, and artifacts. Fourth, it lays the groundwork for future expansion, as the identical data processing can be used to serve additional subjects, additional tuned variations, and additional features of the system in the future.

Due to this, the data pipeline is not merely a background process within the project. It is among the principles of the completed system.

Appendix C: Program Source Library, Presentation, and Demonstration

This appendix outlines the major implementation artefacts and support resources applicable to the Personalized Education Agent. It is mainly meant to show how the technical undertakings that come about in relation to the project are organised and how the system can be provided in a comprehensive way when it comes to submission, review, and demonstration. A tight project report is not only an explanation of the architecture and results, but also should be testaments of the underlying work being organized in a systematic way and can be replicated and presented.

The project will include model experimentation notes, preprocess workflows, evaluation resources, adapters, visual materials, and report material, which are all part of the practical implementation layer of the system. The organisation of these parts is explicitly an absolute requirement, as the Personalised education agent is a multi-model platform; without the consistent structure of artefacts, the process of development can be cumbersome and hard to control.

C.1 Program Source Library Overview

The project source library may be separated into some main categories. These will be preprocessed data, data that is clean, data transformed into prompt response pairs, and auxiliary data that is used to organize data and prepare it to be trained and evaluated.

The second group includes artifacts that are associated with model experimentation, such as notebooks and scripts that can be used to load and run a model, fine-tune it in a parameter-efficient way, and verify the inference.

The third category involves evaluation artifacts, which include standard benchmark workflows, an overview of results, runtime metrics, and logs of output review, which will help to perform a comparative analysis of the chosen models.

The fourth category is based on the system design and visualization artifacts, such as architectural diagrams, comparative charts of models, workflow diagrams, schematics, and tabular reports.

All these groups make up the technical library that formed the basis of the workbook.

C.2 Suggested Project Organization

The project artifacts should be arranged into separate parts, such as data, model notebooks, evaluation, figures, and documentation, to increase their clarity and maintainability. This structure will ease the process of review, and further updates will be easy. Organizations that are well structured are especially consequential where there are a number of models and workflows that are in existence.

A suitable organization could include a separate directory of raw and processed information, a different one of model-specific notebooks and training procedures, one more of stored adapters and checkpoints, another of benchmark and analysis results, one of diagrams and figures, and a last one of final report and presentation files. Such a hierarchical structure produces the impression of the project as a professional one and makes it easier to understand.

Folder / Artifact Group	Purpose
Data	Stores raw, cleaned, and formatted datasets used for training and evaluation
Preprocessing Files	Contains scripts or notebooks for data cleaning, formatting, and dataset preparation
Model Notebooks	Includes model-specific experimentation, loading, tuning, and inference workflows
Adapters / Checkpoints	Stores parameter-efficient fine-tuned adapters and saved model checkpoints
Evaluation Outputs	Contains benchmark results, runtime measurements, and validation summaries
Figures	Stores architecture diagrams, charts, comparison visuals, and report images

Tables	Contains structured summary tables used in the workbook
Report Files	Includes the final workbook draft, supporting notes, and submission-ready documents
Presentation Materials	Stores slides, demo visuals, and presentation assets for project review
Logs / Outputs	Contains generated responses, runtime logs, and model output records
Documentation	Maintains project explanations, workflow notes, and supporting technical descriptions

C.3 Notebook and Model Experiment Artifacts

The notebook artifacts are an essential part of the project since they summarize the workflow that is used in practical tasks of experimental research. These include model loading, tokenizer settings, quantised execution, fine training specifications, training chronology, and the generation of results. Since the project compares various specific models, these artifacts will be used to record the evaluation method of each of them to outline the sources of the key results.

These materials are especially important at the stage of reviewing the project as they speak of the fact that the report is supported by empirical experimentation, but not only by the comparison of the concepts. Additionally, they also improve reproducibility in the sense that future researchers can rely on the same model workflows and have access to the same experimental background.

C.4 Evaluation and Result Artifacts

The project also includes the artifacts of evaluation that support the main analytical parts of the report. These artifacts include benchmark performance, runtime measurements, training monitors, and examples of system behavior. In the final project structure, these outputs should be well-organized so that the tables, charts, and interpretations, which are provided in the report, can be easily connected with the model paths.

This type is important because it allows connecting the experimental stage with the written conclusions. It also eases professional presentation as charts and summaries of results can easily be reused in the workbook as well as the project demonstration.

C.5 Presentation Structure

A convincing and stringent display of the Personalized Education Agent should reflect the logical flow that is used in the related report. The introduction is to state the problem statement and explain the motivation behind the selected models, then outline the selected models, summarize the methodology of comparing and evaluating the models, system architecture, and eventually explain the intelligent routing mechanism and the interesting results. This methodology congruence makes it easy to understand and have consistency between the presentation and the written workbook.

The best parts of the presentation will include the high-level architecture diagram, routing workflow schematic, the model comparison matrix, the benchmark summary chart, the quality-versus-efficiency analytical graph, and at least one example tutoring interaction. Such artifacts must be in line with what is provided in the report in such a way that the presentation and the workbook reinforce one narrative of the project.

C.6 Demonstration Plan

The next step in presenting the project should focus on the mechanics of operation of the system as a form of learning, by avoiding a simple list of model names. A clear demonstration chain should start with a sample student query, outline the process of how the system is able to identify the type of task being done, how the routing layer is able to identify the correct model pathway, and end with the display of the final tutoring response.

To ensure the strength of the demonstration, it is worthwhile to include examples of several types of tutoring. A conceptual explanation, a coding assistance scenario, a debugging instance, and an advanced reasoning case, respectively, are all examples of how the platform can support the whole range of intended use cases. With this thorough demonstration, the evaluators will be able to arrive at the conclusion that the project is a structured tutoring system, but not a comparison of models.

C.7 Demonstration Readiness

The project has already shown willingness to demonstrate due to its well-written technical story, outlined model plan, detailed system architecture, and systematic assessment plan. Even though the current stage of the project focuses on the architectural polishing, proven working processes rather than large-scale production implementation, the system can be conveyed through its diagrammatic forms, example cases, and artefacts organized consistently.

This ability is one of the practical strengths of the project. The workbook, model experiments, and visuals all contribute to the overall claim that Personalized Education Agent is a multi-model intelligent tutoring system that has been designed to provide various types of academic assistance in a structured and adaptive arrangement.

C.8 Appendix Summary

The next step in the project presentation will need to focus on the working mechanism of the system and not just list the models that form the system. A representative student query should start a systematic demonstration process that outlines the process through which the system recognizes the type of task, shows how the routing layer picks the right model pathway, and ends with the expression of the end tutoring response.

As a way of strengthening the rigor of the demonstration, it is prudent to have exemplars in various tutoring modalities. The examples of conceptual explanation, code support, debugging, and advanced reasoning can all be demonstrated to demonstrate the ability of the platform to support the entire range of use cases that are desired. This detailed illustration will support the fact that the project is a well-organized tutoring system, not just a review of the underlying models.

LINKS

1. Drive link consisting of the Fine-Tuning Notebooks:

<https://drive.google.com/drive/folders/16yBd2y6beAm7SkFz7Rf1Imr0-vdITyh5?usp=sharing>

2. Dataset:

<https://huggingface.co/datasets/HuggingFaceTB/cosmopedia>

3. Data Stored in S3

<https://s3.console.aws.amazon.com/s3/buckets/education-agent-nischitha-dev>

4. System architecture link :

https://drive.google.com/file/d/16v9oy2Z0IfjYzCQb_jjfe9MUiIPkay7_/view

5. Pptlink :

<https://gamma.app/docs/Personalized-Education-Agent-h5xupbo938zapt4>